PhysicalAI応用編1 最終課題 解説記事シリーズ 目次

ROS講座（https://qiita.com/srs/items/9a1d3bc6f3d381c4d40b ）の形式（環境 → 概要 → ソースコード → 実行 → 参考）にならい、最終課題の候補20件について、動かし方と作り方をまとめています。

このシリーズの前提

- コマンドとコードは各リポジトリの公式README・公式ドキュメントをもとに書いています。ただし、筆者の環境で一つずつ動作確認したものではありません。ライブラリの版によって引数名やAPI名が変わることがあるので、エラーが出たら公式READMEの最新版と照らし合わせてください。
- 「工夫ポイント」の節は、最終課題で丸写しにならないための発展案です。レポートの「工夫した点」にそのまま使えるように書いています。

共通の環境

- OS: Ubuntu 22.04
- Windows 11の場合は WSL2 + Ubuntu 22.04 を推奨します。GazeboのGUIはWSLgで表示できます。
- ROS: ROS 2 Humble
- シミュレータ（記事によって使い分けます）
- Gazebo Classic 11（TurtleBot3系）
- Gazebo Fortress（UR系）
- Genesis
- MuJoCo / PyBullet
- Python: 3.10（ACTとCLIPortは3.8）
- GPU: NVIDIA RTX 3060（12GB）以上を推奨
- GPUがない場合、Genesisの記事は Google Colab でも動かせます（PDFのサンプル「google colabを利用して自前GPUなしでGenesisの強化学習を試す」を参照）。

A. Qiita・note・GitHubの代表的な実装

- A01 TurtleBot3がYOLOで物体を見つけて追従する（初級）
- A02 UR + Gazebo + MoveIt2 で物体を指定して手先を移動させる（中級）
- A03 Genesisで4脚ロボットGo2の歩行を強化学習する（中〜上級）
- A04 LeRobotで模倣学習（ACT / Diffusion Policy）をシミュレーションで試す（中級）
- A05 SO-101実機でLeRobotの模倣学習をする（中級・実機必要）
- A06 SmolVLAで言語指示によるアーム操作（中〜上級）
- A07 Gemini Robotics-ER 2 で机の上を片付ける（Genesis + Franka）（中級）
- この記事の「共通部品 franka_api.py」は B05・B06・B07 でも使います。
- A08 ROSA（NASA JPL）で自然言語からROS2ロボットを操作する（中級）
- A09 Ultralytics YOLOを自作データで学習させる（初級）
- A10 停止標識と歩行者を認識して止まる自動走行ロボット（初級）

B. 論文とGitHubコードが公開されているもの

- B01 ACT（ALOHA）を公式コードで再現する
- B02 Diffusion PolicyをPushTで再現し、ACTと比較する

- B03 legged_gymの地形カリキュラムをGenesisに移植する
- B04 Eurekaの仕組みでLLMに報酬関数を書かせる
- B05 Code as Policies: LLMが書いたコードで積み木を積む
- B06 SayCan: LLMのスコアと実行可能性を掛け合わせて行動を選ぶ
- B07 VoxPoser: LLMが作る3D価値マップで障害物を避けて動く
- B08 CLIPortで言語指示による積み替えと未知色への汎化を評価する
- B09 OpenVLAをLIBEROで評価する
- B10 $\pi 0$（openpi）をLIBEROで動かしてOpenVLAと比較する

提出動画の作り方（全記事共通）

- 画面録画には OBS Studio を使います。Genesis の記事では、カメラの録画機能でmp4を直接出力できます。
- 2倍速にするコマンド（倍速にした場合は、動画内か概要欄にその旨を明記してください）：

```
ffmpeg -i input.mp4 -filter:v "setpts=0.5*PTS" -an output_2x.mp4
```

- 1〜3分に収める構成例: 1. タイトルと課題の目的（5秒） 2. システム構成図（10秒） 3. 実行の様子（字幕付き） 4. 比較や定量評価の結果 5. まとめ

A01 TurtleBot3がYOLOで物体を見つけて追従する

環境

- OS: Ubuntu 22.04（WSL2可）
- ROS: ROS 2 Humble
- シミュレータ: Gazebo Classic 11
- ロボット: TurtleBot3 Waffle Pi（カメラ付きのモデル）
- AI: Ultralytics YOLO11n（推論のみ）
- 参考リポジトリ: https://github.com/ROBOTIS-GIT/turtlebot3_applications
- `turtlebot3_yolo_object_detection` と `turtlebot3_follower` が含まれています。

概要

Gazeboの中のTurtleBot3が部屋を探索します。カメラ画像にYOLOをかけて、指定した物体（例: `person`）を見つけたら、その方向に向きを合わせながら近づき、一定の距離まで来たら止まります。

処理の流れは次のとおりです。

1. `/camera/image_raw`（Image）を受け取る。
2. `yolo_follower` ノードがYOLOで推論する。
3. 状態機械で SEARCH（その場で回転して探す）→ TRACK（追従）→ ARRIVED（停止）を切り替える。
4. `/cmd_vel`（Twist）を出してTurtleBot3を動かす。
5. デバッグ用に、検出枠を描いた画像 `/yolo/debug_image` も出す。

今回のポイントは次の3つです。

- 検出枠の**横方向のずれ**を角速度に変える（P制御）。
- 検出枠の**画面に占める面積の割合**を、距離の代わりの値として使う。
- 見失ったら一定時間待ってから探索に戻る（ちらつき対策）。

準備

1. ROS 2 Humble と Gazebo のインストール

ROS 2 Humble 本体は公式手順（https://docs.ros.org/en/humble/Installation/Ubuntu-Install-Debs.html ）で `ros-humble-desktop` まで入れておきます。そのうえで次を実行します。

```
sudo apt update
sudo apt install -y ros-humble-gazebo-ros-pkgs ros-humble-cartographer ros-humble-cartographer-ros \
  ros-humble-navigation2 ros-humble-nav2-bringup ros-humble-cv-bridge python3-colcon-common-extensions
```

2. TurtleBot3 シミュレーションのビルド

```
mkdir -p ~/turtlebot3_ws/src && cd ~/turtlebot3_ws/src
git clone -b humble https://github.com/ROBOTIS-GIT/DynamixelSDK.git
git clone -b humble https://github.com/ROBOTIS-GIT/turtlebot3_msgs.git
git clone -b humble https://github.com/ROBOTIS-GIT/turtlebot3.git
```

```
git clone -b humble https://github.com/ROBOTIS-GIT/turtlebot3_simulations.git
cd ~/turtlebot3_ws
colcon build --symlink-install
echo 'source ~/turtlebot3_ws/install/setup.bash' >> ~/.bashrc
echo 'export TURTLEBOT3_MODEL=waffle_pi' >> ~/.bashrc
echo 'export GAZEBO_MODEL_PATH=$GAZEBO_MODEL_PATH:~/turtlebot3_ws/src/turtlebot3_simulations/turtlebot3_gazebo/models' >
> ~/.bashrc
source ~/.bashrc
```

- 古い記事ではブランチ名が `humble-devel` になっています。clone でエラーになる場合は `git branch -r` でブランチ名を確認してください。
- カメラが付いているのは `waffle_pi` だけです。`burger` にはカメラがないので注意してください。

3. YOLO のインストール

```
pip install ultralytics
pip install "numpy<2"
```

ultralytics を入れると numpy 2.x に上がることがあります。そうなると Humble の `cv_bridge` が `numpy.core.multiarray failed to import` で落ちるので、numpy を 1.x に戻しておきます。

4. 動作確認

```
ros2 launch turtlebot3_gazebo turtlebot3_world.launch.py
```

別の端末でトピックを確認します。

```
ros2 topic list | grep camera
# /camera/image_raw が出ればOK
```

ソースコード

パッケージ作成

```
cd ~/turtlebot3_ws/src
ros2 pkg create --build-type ament_python yolo_follower \
  --dependencies rclpy sensor_msgs geometry_msgs cv_bridge
```

yolo_follower/yolo_follower/follower_node.py

```
import rclpy
from rclpy.node import Node
from sensor_msgs.msg import Image
from geometry_msgs.msg import Twist
from cv_bridge import CvBridge
from ultralytics import YOLO
```

```
class YoloFollower(Node):
    def __init__(self):
        super().__init__('yolo_follower')
        # パラメータ（launch や --ros-args で変更可能）
        self.declare_parameter('target', 'person')
        self.declare_parameter('model', 'yolo11n.pt')
        self.declare_parameter('conf', 0.5)
        self.declare_parameter('stop_area_ratio', 0.15)
        self.declare_parameter('kp_angular', 0.8)
        self.declare_parameter('linear_speed', 0.15)

        self.target = self.get_parameter('target').value
        self.conf = self.get_parameter('conf').value
        self.stop_ratio = self.get_parameter('stop_area_ratio').value
        self.kp = self.get_parameter('kp_angular').value
        self.v = self.get_parameter('linear_speed').value

        self.model = YOLO(self.get_parameter('model').value)
        self.bridge = CvBridge()

        self.sub = self.create_subscription(Image, '/camera/image_raw', self.on_image, 10)
        self.pub_cmd = self.create_publisher(Twist, '/cmd_vel', 10)
        self.pub_dbg = self.create_publisher(Image, '/yolo/debug_image', 10)

        self.state = 'SEARCH'
        self.lost_count = 0
        self.get_logger().info(f'target = {self.target}')

    def find_target(self, result):
        """target クラスの中で一番大きい（=近い）検出枠を返す"""
        best = None
        for box in result.boxes:
            name = result.names[int(box.cls)]
            if name != self.target or float(box.conf) < self.conf:
                continue
            x1, y1, x2, y2 = box.xyxy[0].tolist()
            area = (x2 - x1) * (y2 - y1)
            if best is None or area > best['area']:
                best = {'cx': (x1 + x2) / 2, 'area': area}
        return best

    def on_image(self, msg):
        frame = self.bridge.imgmsg_to_cv2(msg, 'bgr8')
        h, w = frame.shape[:2]
        result = self.model(frame, verbose=False)[0]
        target = self.find_target(result)

        cmd = Twist()
        if target is None:
```

```
            self.lost_count += 1
            # 10フレーム連続で見失ったら探索に戻る
            if self.lost_count > 10:
                self.state = 'SEARCH'
            if self.state == 'SEARCH':
                cmd.angular.z = 0.4
        else:
            self.lost_count = 0
            err_x = (target['cx'] - w / 2) / (w / 2)   # -1(左) 〜 +1(右)
            ratio = target['area'] / (w * h)
            cmd.angular.z = -self.kp * err_x
            if ratio < self.stop_ratio:
                self.state = 'TRACK'
                cmd.linear.x = self.v
            else:
                self.state = 'ARRIVED'

        self.pub_cmd.publish(cmd)

        # デバッグ画像（検出枠 + 状態）
        dbg = result.plot()
        self.pub_dbg.publish(self.bridge.cv2_to_imgmsg(dbg, 'bgr8'))
        self.get_logger().info(
            f'state={self.state} v={cmd.linear.x:.2f} w={cmd.angular.z:.2f}',
            throttle_duration_sec=1.0)


def main():
    rclpy.init()
    node = YoloFollower()
    try:
        rclpy.spin(node)
    except KeyboardInterrupt:
        pass
    node.pub_cmd.publish(Twist())  # 終了時は停止させる
    node.destroy_node()
    rclpy.shutdown()
```

ここから細かく解説します。

- `find_target`
- 同じクラスの物体が複数見えている場合は、検出枠が一番大きいもの（=一番近いもの）を選びます。
- **角速度の計算**
- `err_x` は、画面中央からのずれを -1〜1 に正規化した値です。
- ROS では反時計回り（左回り）が正の角速度なので、対象が右（`err_x > 0`）にあるときは負の角速度で右を向きます。そのため符号はマイナスになります。
- **停止の判定**
- 検出枠が画面の15%を占めたら到着とみなします。

- 距離センサを使わずにカメラだけで止められますが、物体の実際の大きさによって止まる距離が変わります。工夫ポイントの「LiDARとの融合」で改善できます。
- `lost_count`
- YOLO は1フレームだけ検出を落とすことがよくあります。すぐに SEARCH に戻ると首を振る動きになってしまうので、10フレーム分は様子を見ます。

setup.py（entry_points 部分）

```
    entry_points={
        'console_scripts': [
            'follower_node = yolo_follower.follower_node:main',
        ],
    },
```

yolo_follower/launch/follower.launch.py

```
from launch import LaunchDescription
from launch.actions import DeclareLaunchArgument
from launch.substitutions import LaunchConfiguration
from launch_ros.actions import Node


def generate_launch_description():
    return LaunchDescription([
        DeclareLaunchArgument('target', default_value='person'),
        Node(
            package='yolo_follower',
            executable='follower_node',
            parameters=[{'target': LaunchConfiguration('target')}],
            output='screen',
        ),
    ])
```

launch フォルダを install に含めるため、setup.py の `data_files` に次の1行を追加します。

```
        ('share/' + package_name + '/launch', ['launch/follower.launch.py']),
```

この行を書くために、setup.py の先頭に `import os` と `from glob import glob` を入れておくとよいでしょう。今回はファイル名を直接書いているので、なくても動きます。

ビルドと実行

```
cd ~/turtlebot3_ws
colcon build --symlink-install --packages-select yolo_follower
source install/setup.bash
```

端末1: シミュレータを起動

```
ros2 launch turtlebot3_gazebo empty_world.launch.py
```

端末1（GazeboのGUI）：追従する対象を置く

Gazebo の左側の Insert タブから `Person Standing` を選び、ロボットの数m先に置きます。モデル一覧が出てこない場合は、`~/.gazebo/models` に gazebo_models（https://github.com/osrf/gazebo_models ）を clone してください。

端末2: 追従ノードを起動

```
ros2 launch yolo_follower follower.launch.py target:=person
```

端末3: デバッグ画像を表示

```
ros2 run rqt_image_view rqt_image_view /yolo/debug_image
```

実行結果の見方

- 起動直後は、ロボットがその場で回転します（SEARCH）。
- 人が画面に入ると、枠が中央に来るように向きを変えながら前進します（TRACK）。
- 近づくと止まります（ARRIVED）。
- GazeboのGUIで人を掴んで動かすと、再び追従を始めます。

工夫ポイント（最終課題として発展させる案）

1. **LiDARとの融合** - `/scan`（LaserScan）から、カメラの向いている方向の距離を取り、「0.5mで停止」のように実際の距離で止める。 - 検出枠の面積で止める方式と比較すると、レポートに書く材料になります。
2. **見失ったときの賢い探索** - 最後に見えた方向（`err_x` の符号）に向かって回転する。
3. **複数の対象と優先順位** - 例: `person` を最優先し、`sports ball` を2番目にする。
4. **Nav2との組み合わせ** - 探索を「その場で回転」ではなく、Nav2 で部屋の巡回地点を順番に回る方式にする。 - `turtlebot3_world` や `turtlebot3_house` で行うと、見栄えが良くなります。
5. **定量評価** - 対象を5か所に置いたときの「発見までの時間」と「停止距離の誤差」を表にする。

参考

- TurtleBot3 e-Manual（Simulation）: https://emanual.robotis.com/docs/en/platform/turtlebot3/simulation/
- turtlebot3_applications: https://github.com/ROBOTIS-GIT/turtlebot3_applications
- turtlebot3_yolo_object_detection（ROS Index）: https://index.ros.org/p/turtlebot3_yolo_object_detection
- Ultralytics Docs: https://docs.ultralytics.com/
- Redmon et al., "You Only Look Once: Unified, Real-Time Object Detection", CVPR 2016

目次へ

A02 UR + Gazebo + MoveIt2 で物体を指定して手先を移動させる

環境

- OS: Ubuntu 22.04
- ROS: ROS 2 Humble
- シミュレータ: Gazebo Fortress（ros_gz）
- ロボット: Universal Robots UR5e（`ur_simulation_gz`）
- 動作計画: MoveIt2、および Python から MoveIt2 を呼ぶライブラリ pymoveit2
- AI: Ultralytics YOLO-World（物体名を文字で指定できるオープンボキャブラリ検出。推論のみ）
- 参考リポジトリ:
- https://github.com/UniversalRobots/Universal_Robots_ROS2_GZ_Simulation
- https://github.com/AndrejOrsula/pymoveit2

概要

床に置いた色付きの箱を、真上から見下ろすカメラで撮影します。「red box」のように物体名を文字で指定すると、YOLO-World がその物体を検出し、URアームの手先（tool0）がその真上まで移動します。続けて、トレイの位置へ運ぶ動作をします。

処理の流れは次のとおりです。

1. Gazeboの真上カメラが画像を出す。
2. ros_gz_bridge が `/overhead_camera` として ROS に流す。
3. `pick_by_name` ノードで次の順に処理する。
4. YOLO-World で「red box」を検出する。
5. 画素座標から床面の座標に変換する（ホモグラフィ）。
6. pymoveit2 で「物体の真上 → 下降 → 上昇 → トレイの上」と手先を動かす。
7. MoveIt2 が計画した軌道を UR5e が実行する。

注意: `ur_simulation_gz` の初期状態にはグリッパが付いていません。この記事では、物体の真上まで手先を移動させる部分（認識と動作計画）までを作ります。実際に掴ませたい場合は、工夫ポイントの1を参照してください。

準備

1. パッケージのインストール

```
sudo apt update
sudo apt install -y ros-humble-ur ros-humble-ur-simulation-gz ros-humble-ros-gz \
  ros-humble-moveit ros-humble-cv-bridge
pip install ultralytics "numpy<2"
```

`ros-humble-ur-simulation-gz` が apt で見つからない場合は、ソースからビルドします。

```
mkdir -p ~/ur_ws/src && cd ~/ur_ws/src
git clone -b humble https://github.com/UniversalRobots/Universal_Robots_ROS2_GZ_Simulation.git
cd ~/ur_ws && rosdep install --from-paths src --ignore-src -r -y && colcon build --symlink-install
```

2. pymoveit2 のビルド

```
mkdir -p ~/ur_ws/src && cd ~/ur_ws/src
git clone https://github.com/AndrejOrsula/pymoveit2.git
cd ~/ur_ws
rosdep install -y -r -i --rosdistro humble --from-paths src
colcon build --merge-install --symlink-install --cmake-args "-DCMAKE_BUILD_TYPE=Release"
source install/setup.bash
```

3. 動作確認（MoveIt2付きで起動）

```
ros2 launch ur_simulation_gz ur_sim_moveit.launch.py ur_type:=ur5e
```

- Gazebo と RViz が立ち上がり、RViz の MotionPlanning パネルで Plan & Execute ができればOKです。
- 起動引数の一覧は `ros2 launch ur_simulation_gz ur_sim_moveit.launch.py --show-args` で確認できます。次の手順で `world_file` 引数を使うので、名前を確認しておいてください。

ソースコード

ワールド: ~/ur_ws/worlds/pick_world.sdf

真上カメラと色付きの箱3つ、トレイを置いたワールドです。カメラを使うために、**Sensors システムプラグイン**を必ず読み込みます。

```
<?xml version="1.0" ?>
<sdf version="1.8">
  <world name="pick_world">
    <plugin filename="ignition-gazebo-physics-system" name="ignition::gazebo::systems::Physics"/>
    <plugin filename="ignition-gazebo-user-commands-system" name="ignition::gazebo::systems::UserCommands"/>
    <plugin filename="ignition-gazebo-scene-broadcaster-system" name="ignition::gazebo::systems::SceneBroadcaster"/>
    <plugin filename="ignition-gazebo-sensors-system" name="ignition::gazebo::systems::Sensors">
      <render_engine>ogre2</render_engine>
    </plugin>

    <light type="directional" name="sun">
      <pose>0 0 10 0 0 0</pose>
      <diffuse>1 1 1 1</diffuse>
      <direction>-0.3 0.2 -0.9</direction>
    </light>

    <model name="ground_plane">
      <static>true</static>
      <link name="link">
        <collision name="c"><geometry><plane><normal>0 0 1</normal><size>10 10</size></plane></geometry></collision>
        <visual name="v"><geometry><plane><normal>0 0 1</normal><size>10 10</size></plane></geometry>
          <material><diffuse>0.8 0.8 0.8 1</diffuse></material></visual>
      </link>
    </model>
```

```
<!-- 真上から見下ろすカメラ（pitch=90deg で下向き） -->
<model name="overhead_camera">
  <static>true</static>
  <pose>0.45 0 1.6 0 1.5708 0</pose>
  <link name="link">
    <sensor name="cam" type="camera">
      <topic>overhead_camera</topic>
      <update_rate>10</update_rate>
      <camera>
        <horizontal_fov>1.0</horizontal_fov>
        <image><width>640</width><height>480</height></image>
        <clip><near>0.1</near><far>10</far></clip>
      </camera>
    </sensor>
  </link>
</model>

<!-- 色付きの箱（5cm角） -->
<model name="red_box">
  <pose>0.45 0.20 0.025 0 0 0</pose>
  <link name="link">
    <inertial><mass>0.1</mass></inertial>
    <collision name="c"><geometry><box><size>0.05 0.05 0.05</size></box></geometry></collision>
    <visual name="v"><geometry><box><size>0.05 0.05 0.05</size></box></geometry>
      <material><diffuse>1 0 0 1</diffuse></material></visual>
  </link>
</model>
<model name="blue_box">
  <pose>0.55 -0.15 0.025 0 0 0</pose>
  <link name="link">
    <inertial><mass>0.1</mass></inertial>
    <collision name="c"><geometry><box><size>0.05 0.05 0.05</size></box></geometry></collision>
    <visual name="v"><geometry><box><size>0.05 0.05 0.05</size></box></geometry>
      <material><diffuse>0 0 1 1</diffuse></material></visual>
  </link>
</model>
<model name="green_box">
  <pose>0.35 -0.05 0.025 0 0 0</pose>
  <link name="link">
    <inertial><mass>0.1</mass></inertial>
    <collision name="c"><geometry><box><size>0.05 0.05 0.05</size></box></geometry></collision>
    <visual name="v"><geometry><box><size>0.05 0.05 0.05</size></box></geometry>
      <material><diffuse>0 1 0 1</diffuse></material></visual>
  </link>
</model>

<!-- トレイ（置き場所） -->
<model name="tray">
```

```
      <static>true</static>
      <pose>0.0 0.45 0.005 0 0 0</pose>
      <link name="link">
        <visual name="v"><geometry><box><size>0.2 0.2 0.01</size></box></geometry>
          <material><diffuse>0.6 0.4 0.2 1</diffuse></material></visual>
      </link>
    </model>
  </world>
</sdf>
```

ポイントは次のとおりです。

- `ignition-gazebo-sensors-system`
- これがないとカメラは画像を出しません。最初はここでつまずく人が多いです。
- **カメラの姿勢**
- `<pose>0.45 0 1.6 0 1.5708 0</pose>` で、高さ1.6mから真下を向けています。
- **プラグイン名**
- Fortress では `ignition-gazebo-...` です。Garden 以降は `gz-sim-...` に名前が変わります。

pick_by_name/pick_by_name/pick_node.py

```
import time
from threading import Thread

import cv2
import numpy as np
import rclpy
from cv_bridge import CvBridge
from rclpy.callback_groups import ReentrantCallbackGroup
from rclpy.node import Node
from sensor_msgs.msg import Image
from ultralytics import YOLOWorld
from pymoveit2 import MoveIt2

UR_JOINTS = ['shoulder_pan_joint', 'shoulder_lift_joint', 'elbow_joint',
             'wrist_1_joint', 'wrist_2_joint', 'wrist_3_joint']
DOWN_QUAT = [1.0, 0.0, 0.0, 0.0]   # tool0 の z 軸が真下を向く姿勢（x軸まわり180度）
TRAY_XY = (0.0, 0.45)
APPROACH_Z = 0.25     # 物体の上空
TOUCH_Z = 0.10        # 下降したときの tool0 の高さ


class PickByName(Node):
    def __init__(self):
        super().__init__('pick_by_name')
        self.declare_parameter('target', 'red box')
        self.declare_parameter('homography', '')   # calib.npy のパス
        self.bridge = CvBridge()
        self.latest = None
```

```
        cb = ReentrantCallbackGroup()
        self.create_subscription(Image, '/overhead_camera', self.on_image, 10, callback_group=cb)

        self.model = YOLOWorld('yolov8s-world.pt')
        self.model.set_classes(['red box', 'blue box', 'green box'])

        self.moveit2 = MoveIt2(
            node=self, joint_names=UR_JOINTS, base_link_name='base_link',
            end_effector_name='tool0', group_name='ur_manipulator', callback_group=cb)
        self.moveit2.max_velocity = 0.3
        self.moveit2.max_acceleration = 0.3

        h_path = self.get_parameter('homography').value
        self.H = np.load(h_path) if h_path else None

    def on_image(self, msg):
        self.latest = self.bridge.imgmsg_to_cv2(msg, 'bgr8')

    def detect(self, name):
        """name に一致する検出枠の中心画素 (u, v) を返す"""
        result = self.model(self.latest, verbose=False)[0]
        cv2.imwrite('/tmp/detect.png', result.plot())   # レポート用に保存
        for box in result.boxes:
            if result.names[int(box.cls)] == name and float(box.conf) > 0.2:
                x1, y1, x2, y2 = box.xyxy[0].tolist()
                return (x1 + x2) / 2, (y1 + y2) / 2
        return None

    def pixel_to_world(self, u, v):
        p = self.H @ np.array([u, v, 1.0])
        return p[0] / p[2], p[1] / p[2]

    def move(self, x, y, z):
        self.get_logger().info(f'move to ({x:.3f}, {y:.3f}, {z:.3f})')
        self.moveit2.move_to_pose(position=[x, y, z], quat_xyzw=DOWN_QUAT)
        self.moveit2.wait_until_executed()

    def run(self):
        target = self.get_parameter('target').value
        while self.latest is None:
            time.sleep(0.1)
        uv = self.detect(target)
        if uv is None:
            self.get_logger().error(f'{target} が見つかりません')
            return
        x, y = self.pixel_to_world(*uv)
        self.get_logger().info(f'{target}: pixel={uv} -> world=({x:.3f}, {y:.3f})')
        self.move(x, y, APPROACH_Z)
        self.move(x, y, TOUCH_Z)          # ここでグリッパを閉じる（工夫ポイント1）
```

```
        self.move(x, y, APPROACH_Z)
        self.move(*TRAY_XY, APPROACH_Z)   # ここでグリッパを開く


def main():
    rclpy.init()
    node = PickByName()
    executor = rclpy.executors.MultiThreadedExecutor(2)
    executor.add_node(node)
    Thread(target=executor.spin, daemon=True).start()
    node.run()
    rclpy.shutdown()
```

ここから細かく解説します。

- **YOLO-World**
- `set_classes()` に文字列を渡すだけで、そのクラスを検出できるようになります。COCOの80クラスに「赤い箱」はないので、通常のYOLOではなくこちらを使います。
- **`MoveIt2(...)` の引数**
- URのMoveIt設定の名前です。
    - 計画グループ: `ur_manipulator`
    - 手先リンク: `tool0`
    - 基準リンク: `base_link`
- RViz の MotionPlanning パネルでも同じ名前を確認できます。
- **MultiThreadedExecutor を別スレッドで回す**
- pymoveit2 の `wait_until_executed()` は、ROSのコールバックが回っていないと戻ってきません。そのため、spin を別スレッドで回しています。
- これは pymoveit2 の examples と同じ書き方です。
- **`pixel_to_world`**
- 床は平面なので、画素座標から床面の座標への変換は3×3の行列（ホモグラフィ）1つで表せます。
- 行列は次のキャリブレーションで求めます。

pick_by_name/pick_by_name/calibrate.py（ホモグラフィを求める）

ワールドに置いた3つの箱とトレイ、計4点の「既知の床面座標」と「画像上の検出位置」を対応させて、ホモグラフィを求めます。

```
import cv2
import numpy as np
import rclpy
from cv_bridge import CvBridge
from sensor_msgs.msg import Image
from ultralytics import YOLOWorld

# ワールドSDFに書いた座標（base_link 座標系。後述の注意も参照）
KNOWN = {'red box': (0.45, 0.20), 'blue box': (0.55, -0.15),
         'green box': (0.35, -0.05), 'brown tray': (0.0, 0.45)}

rclpy.init()
```

```
node = rclpy.create_node('calib')
bridge = CvBridge()
img = {}
node.create_subscription(Image, '/overhead_camera',
                         lambda m: img.setdefault('f', bridge.imgmsg_to_cv2(m, 'bgr8')), 10)
while 'f' not in img:
    rclpy.spin_once(node, timeout_sec=0.5)

model = YOLOWorld('yolov8s-world.pt')
model.set_classes(list(KNOWN.keys()))
res = model(img['f'], verbose=False)[0]
src, dst = [], []
for box in res.boxes:
    name = res.names[int(box.cls)]
    x1, y1, x2, y2 = box.xyxy[0].tolist()
    src.append([(x1 + x2) / 2, (y1 + y2) / 2])
    dst.append(KNOWN[name])
    print(name, src[-1], '->', dst[-1])
H, _ = cv2.findHomography(np.array(src), np.array(dst))
np.save('calib.npy', H)
print(H)
```

- 4点以上が検出されればホモグラフィを計算できます。
- 検出が足りない場合は、ワールドに目印の箱を追加してください。
- または、画像を表示してマウスで点をクリックする方式に変えても構いません。

注意（座標系）：ワールドSDFの座標は Gazebo の world 座標系です。MoveIt の `base_link` 座標系と一致しているかを、RViz の TF 表示で必ず確認してください。

- URの記述では、`base` と `base_link` の間が z軸まわりに180度回転しています。
- ずれている場合は、`KNOWN` の座標を `base_link` 座標系に変換してから使います。

setup.py（entry_points 部分）

```
    entry_points={
        'console_scripts': [
            'pick_node = pick_by_name.pick_node:main',
        ],
    },
```

calibrate.py は entry_points に登録せず、`python3 calibrate.py` で直接実行します。

実行

端末1: 自作ワールドで UR5e + MoveIt2 を起動

```
ros2 launch ur_simulation_gz ur_sim_moveit.launch.py ur_type:=ur5e world_file:=$HOME/ur_ws/worlds/pick_world.sdf
```

端末2: カメラ画像を ROS へブリッジ

```
ros2 run ros_gz_bridge parameter_bridge /overhead_camera@sensor_msgs/msg/Imageignition.msgs.Image
ros2 run rqt_image_view rqt_image_view /overhead_camera   # 映っているか確認
```

端末3: キャリブレーション（最初の1回だけ）

```
cd ~/ur_ws && python3 src/pick_by_name/pick_by_name/calibrate.py
```

端末3: 物体名を指定して実行

```
ros2 run pick_by_name pick_node --ros-args -p target:="red box" -p homography:=$HOME/ur_ws/calib.npy
```

実行結果の見方

- ログに `red box: pixel=(...) -> world=(0.45x, 0.20x)` と出ます。ワールドに書いた座標と数mm〜1cmの誤差で一致すれば成功です。
- アームが赤い箱の真上 → 下降 → 上昇 → トレイの上の順に動きます。
- `/tmp/detect.png` に検出結果の画像が保存されるので、レポートに貼れます。

工夫ポイント（最終課題として発展させる案）

1. **実際に掴む** - 方法A: UR + Robotiq 2F-85 の組み合わせに変える（PDFの中級サンプル「Pick and Place Task (UR3 + Robotiq)」の構成）。 - 方法B: Gazebo の `DetachableJoint` システムプラグインで、「下降したら箱を tool0 に固定し、トレイの上で外す」疑似的な把持をする。
    - 方法Bは物理的な把持ではないので、レポートにはそのことを書いておきます。
2. **自然言語から物体名を抽出する** - 「赤いのをトレイに片付けて」という文章から、LLM（Ollama などのローカルLLM）で `red box` を抽出する。A08 の ROSA と組み合わせることもできます。
3. **未学習の名前でも動くことを示す** - `set_classes` に「tomato」「apple」などを入れて、YCBモデルなどで試す。
4. **定量評価** - 箱の位置をランダムに10回変え、位置推定誤差の平均と成功率を出す。

参考

- Universal Robots ROS2 GZ Simulation: https://github.com/UniversalRobots/Universal_Robots_ROS2_GZ_Simulation
- ur_simulation_gz ドキュメント: https://docs.universal-robots.com/Universal_Robots_ROS2_Documentation/doc/ur_simulation_gz/ur_simulation_gz/doc/installation.html
- pymoveit2: https://github.com/AndrejOrsula/pymoveit2
- YOLO-World（Ultralytics）: https://docs.ultralytics.com/models/yolo-world/
- Cheng et al., "YOLO-World: Real-Time Open-Vocabulary Object Detection", CVPR 2024
- PDFのサンプル: Pick and Place Task - ROS2 Gazebo Simulation (UR3 + Robotiq 2f-85) https://www.youtube.com/watch?v=t4L4VAfgqZw

[目次へ

A03 Genesisで4脚ロボットGo2の歩行を強化学習する

環境

- OS: Ubuntu 22.04（Google Colab の T4 / L4 でも可）
- Python: 3.10
- シミュレータ: Genesis（`genesis-world`）
- 強化学習: PPO（rsl-rl-lib）
- ロボット: Unitree Go2（Genesis に付属するURDF）
- GPU: NVIDIA（VRAM 8GB以上）
- 参考:
- https://github.com/Genesis-Embodied-AI/Genesis （examples/locomotion）
- 公式チュートリアル: https://genesis-world.readthedocs.io/en/latest/user_guide/policy_training/examples/locomotion.html
- 日本語の解説（Zenn）: https://zenn.dev/oyatomo/articles/faf31f476938c9
- PDFのサンプル: [Genesis] 1時間で4脚ロボットの強化学習の環境構築 https://qiita.com/hEnka/items/cc5fd872eb0bf7cd3abc

概要

Genesis の公式サンプルを使い、Go2 が指令速度どおりに歩く方策（policy）を、4096個の環境を並列に動かす強化学習で数分〜数十分かけて学習させます。

この記事では、まず公式どおりに学習と評価を動かします。そのうえで、次の改造を加えて、最終課題として独自性を出す手順を説明します。

- 報酬を追加する
- 外乱（押す力）を加える
- 指令速度の範囲を変える
- 動画を書き出す

ファイル構成は次のとおりです。

- `go2_train.py`
- 設定（env_cfg / obs_cfg / reward_cfg / command_cfg）を作り、PPOで学習します。
- `go2_env.py`
- 環境の本体です。`step()` と `reset_idx()`、それに報酬関数 `_reward_*` が書かれています。
- `go2_eval.py`
- 学習したチェックポイントを読み込み、ビューアで再生します。

準備

1. PyTorch と Genesis のインストール

```
conda create -n genesis python=3.10 -y
conda activate genesis
# CUDA 12.1 の例。自分のCUDAに合わせて https://pytorch.org/ のコマンドを使う
```

```
pip install torch torchvision --index-url https://download.pytorch.org/whl/cu121
pip install genesis-world
```

2. サンプルコードと rsl-rl の取得

```
git clone https://github.com/Genesis-Embodied-AI/Genesis.git
cd Genesis/examples/locomotion
pip install tensorboard
pip install rsl-rl-lib==2.2.4
```

注意: rsl-rl-lib は版によってAPIが変わります。`go2_train.py` の先頭のコメントや、公式チュートリアルに書かれている版を入れてください。`ImportError` や `KeyError: 'class_name'` が出たら、まず版の不一致を疑います。

3. 動作確認

```
python -c "import genesis as gs; gs.init(backend=gs.gpu); print('ok')"
```

実行（公式サンプルどおり）

学習

```
python go2_train.py -e go2-walking -B 4096 --max_iterations 101
```

- `-e`: 実験名です。`logs/go2-walking/` に結果が保存されます。
- `-B`: 並列環境の数です。VRAMが足りない場合は 2048 や 1024 に下げます。
- `--max_iterations`: PPOの反復回数です。101回なら RTX 3060 で数分〜十数分です。

学習曲線の確認

```
tensorboard --logdir logs
```

`Train/mean_reward` が右肩上がりになり、`Train/mean_episode_length` が上限に張り付けば、転ばずに歩けています。

評価（ビューアで再生）

```
python go2_eval.py -e go2-walking --ckpt 100
```

ソースコードの読み方と改造

ここから細かく解説します。行番号は版によって変わるので、関数名で探してください。

1. 設定（go2_train.py の `get_cfgs()`）

`get_cfgs()` は、次の4つの辞書を返します。

- `env_cfg`
- 関節の名前、初期姿勢、PDゲイン（`kp`, `kd`）、エピソード長、行動のスケールなどです。
- `obs_cfg`

- 観測の次元とスケールです。
- `reward_cfg`
- 報酬のスケールです。キー名が `go2_env.py` の `_reward_<キー名>` 関数に対応します。
- `command_cfg`
- 指令速度の範囲です（`lin_vel_x_range` など）。

`reward_cfg["reward_scales"]` には、たとえば次のようなものが並んでいます。

- `tracking_lin_vel`：指令した並進速度に追従する（正の報酬）
- `tracking_ang_vel`：指令した回転速度に追従する（正の報酬）
- `lin_vel_z`：上下に跳ねることへのペナルティ（負の報酬）
- `base_height`：胴体の高さを保つ
- `action_rate`：行動の急な変化へのペナルティ
- `similar_to_default`：初期姿勢から離れすぎることへのペナルティ

改造1: 指令速度の範囲を広げる（後ろ歩きと旋回も学ばせる）

```
command_cfg = {
    "num_commands": 3,
    "lin_vel_x_range": [-0.5, 1.0],   # 元は前進のみ（例: [0.5, 0.5]）
    "lin_vel_y_range": [-0.3, 0.3],
    "ang_vel_range": [-1.0, 1.0],
}
```

元のサンプルは指令が固定値に近いので、まっすぐ歩くことしか学びません。範囲を広げると、方向を指定して歩けるようになり、デモの見栄えが大きく変わります。その分学習は難しくなるので、`--max_iterations` を 300〜500 に増やします。

2. 報酬を追加する（go2_env.py）

`go2_env.py` の中では、`reward_scales` のキーごとに `self._reward_<キー名>` が呼ばれ、スケールを掛けて足し合わされます。そのため、**関数を1つ追加し、`reward_cfg` にキーを1つ足すだけ**で報酬を増やせます。

改造2: 胴体の傾きペナルティを追加する

```
# go2_env.py のクラス内に追加
def _reward_orientation(self):
    # 重力ベクトルを胴体座標で見たときの xy 成分 = 胴体の傾き
    return torch.sum(torch.square(self.projected_gravity[:, :2]), dim=1)
```

```
# go2_train.py の reward_cfg["reward_scales"] に追加
    "orientation": -5.0,
```

- `self.projected_gravity` は観測の計算にも使われている変数です。
- 胴体が水平なら `(0, 0, -1)` になるので、xy成分の2乗和が傾きの大きさを表します。

改造3: 足を上げることへの報酬（足が地面を擦る歩き方の改善）

足先リンクの高さを取得する方法は Genesis の版によって違います。`self.robot.get_links_pos()` が使える版では、次のように書けます。

```
    def _reward_foot_clearance(self):
        # 足先リンクのインデックスは __init__ で
        # self.feet_idx = [self.robot.get_link(n).idx_local for n in ["FL_foot", "FR_foot", "RL_foot", "RR_foot"]]
        # のように用意しておく
        feet_z = self.robot.get_links_pos()[:, self.feet_idx, 2]
        return torch.sum(torch.clamp(feet_z - 0.02, max=0.08), dim=1)
```

3. 外乱を加える（ロバスト性の向上）

一定間隔で胴体に横向きの速度を与え、「押されても転ばない」歩行を学ばせます。`step()` の最後のほうに追加します。

```
        # go2_env.py の step() 内、報酬計算の後あたりに追加
        if self.env_cfg.get("push_interval_s", 0) > 0:
            push_every = int(self.env_cfg["push_interval_s"] / self.dt)
            if self.common_step_counter % push_every == 0:   # カウンタがなければ自分で用意する
                vel = self.robot.get_dofs_velocity()          # 先頭6自由度がベース（並進3 + 回転3）
                vel[:, 0:2] += gs_rand_float(-1.0, 1.0, (self.num_envs, 2), self.device)
                self.robot.set_dofs_velocity(vel)
```

```
    # go2_train.py の env_cfg に追加
        "push_interval_s": 5.0,
```

`gs_rand_float` は `go2_env.py` の中で定義されている乱数関数です。ベースの自由度の並び順は版によって違う可能性があるので、`print(self.robot.dofs_info)` などで確認してください。

4. 動画を書き出す（提出用）

ビューアの画面録画でも構いませんが、Genesis のカメラ機能を使うと mp4 を直接出力できます。Colab ではこちらの方法しか使えません。`go2_eval.py` で、シーンを作る部分（`scene.build()` の前）にカメラを追加します。

```
cam = env.scene.add_camera(res=(1280, 720), pos=(2.0, -2.0, 1.2),
                           lookat=(0.0, 0.0, 0.3), fov=40, GUI=False)
```

`go2_eval.py` では `Go2Env(...)` の中で `scene.build()` が呼ばれるので、実際には `go2_env.py` の `__init__` で `self.cam = self.scene.add_camera(...)` を `self.scene.build(...)` の直前に入れます。評価ループは次のように書きます。

```
env.cam.start_recording()
obs, _ = env.reset()
with torch.no_grad():
    for i in range(1000):
        actions = policy(obs)
        obs, rews, dones, infos = env.step(actions)
        # ロボットを追いかけるカメラ
        base = env.base_pos[0].cpu().numpy()
        env.cam.set_pose(pos=(base[0] + 2.0, base[1] - 2.0, 1.2), lookat=base)
        env.cam.render()
env.cam.stop_recording(save_to_filename="go2_walk.mp4", fps=50)
```

- 評価時は `-B` を 1 にする（`go2_eval.py` は `num_envs=1` で作っています）と、カメラに映るのが1体になります。
- 評価時に指令速度を変えたい場合は、`env.commands[:] = torch.tensor([...])` で上書きしてから `step` を回します。ループの途中で切り替えると、「前進 → 旋回 → 後退」のデモになります。

実行（改造後）

```
python go2_train.py -e go2-robust -B 4096 --max_iterations 500
python go2_eval.py -e go2-robust --ckpt 499
```

工夫ポイント（最終課題として発展させる案）

1. **比較実験** - 素の設定（go2-walking）と改造後（go2-robust）で、外乱を加えたときの転倒率を比べます。 - 例: 100 エピソード中、`reset_buf` が早期終了で立った割合。
2. **地形への対応** - B03 の地形カリキュラムと組み合わせ、段差や坂を歩けるようにします。
3. **LLMに報酬を書かせる** - B04 の Eureka 方式で、`_reward_*` を LLM に生成させます。
4. **キー操作** - 評価時にキーボードで `env.commands` を変え、ラジコンのように操縦するデモにします。

参考

- Genesis: https://github.com/Genesis-Embodied-AI/Genesis
- Genesis Locomotion Tutorial: https://genesis-world.readthedocs.io/en/latest/user_guide/policy_training/examples/locomotion.html
- Zenn: Genesis Go2 強化学習 https://zenn.dev/oyatomo/articles/faf31f476938c9
- rsl_rl: https://github.com/leggedrobotics/rsl_rl
- Rudin et al., "Learning to Walk in Minutes Using Massively Parallel Deep Reinforcement Learning", CoRL 2021

目次へ

A04 LeRobotで模倣学習（ACT / Diffusion Policy）をシミュレーションで試す

環境

- OS: Ubuntu 22.04（WSL2可）
- Python: 3.10（LeRobot の README で指定されている版を使ってください）
- ライブラリ: LeRobot（Hugging Face）
- シミュレーション環境:
- gym-aloha: 両腕ALOHAで、キューブの受け渡しや挿入をします（MuJoCo）。
- gym-pusht: T字型のブロックを目標位置に押します（2D）。
- 方策: ACT、Diffusion Policy
- GPU: VRAM 8GB以上（学習する場合）。推論だけならCPUでも動きます。
- 参考リポジトリ: https://github.com/huggingface/lerobot

概要

LeRobot には、人が操作したデモのデータセットと、学習済みの方策が Hugging Face Hub に公開されています。この記事では次の3段階で進めます。

1. 学習済みモデルの評価を動かして、動画を出力する（学習なし、10分程度）。
2. 公開データセットから、自分で ACT を学習させる（数時間程度）。
3. 学習ステップ数や手法を変えて、成功率を比較する（最終課題の本体）。

LeRobot はCLIが頻繁に更新されます。この記事では `lerobot-train` / `lerobot-eval` 形式（2025年以降の形式）で書いています。古い記事にある `python lerobot/scripts/train.py policy=act env=aloha` 形式は、現在は使えません。

準備

1. conda環境と ffmpeg

```
conda create -y -n lerobot python=3.10
conda activate lerobot
conda install -y ffmpeg -c conda-forge
```

2. LeRobot のインストール（シミュレーション環境付き）

```
git clone https://github.com/huggingface/lerobot.git
cd lerobot
pip install -e ".[aloha,pusht]"
```

- `[aloha,pusht]` を付けると、gym-aloha と gym-pusht が一緒に入ります。
- MuJoCo の描画でエラーが出る場合は、`export MUJOCO_GL=egl` を設定します（GPUのある Linux の場合）。GPUがない環境では `osmesa` を使います。

3. Hugging Face へのログイン（データセットの取得用）

```
huggingface-cli login
```

実行1: 学習済みモデルの評価

Diffusion Policy on PushT

```
lerobot-eval \
  --policy.path=lerobot/diffusion_pusht \
  --env.type=pusht \
  --eval.batch_size=10 \
  --eval.n_episodes=10 \
  --policy.device=cuda \
  --policy.use_amp=false
```

ACT on ALOHA（キューブの受け渡し）

```
lerobot-eval \
  --policy.path=lerobot/act_aloha_sim_transfer_cube_human \
  --env.type=aloha \
  --env.task=AlohaTransferCube-v0 \
  --eval.batch_size=5 \
  --eval.n_episodes=10 \
  --policy.device=cuda
```

結果の見方

- `outputs/eval/<日付>/<時刻>_.../` の中に、次のものが出力されます。
- `eval_info.json`：成功率（`pc_success`）や平均報酬が入っています。
- `videos/`：各エピソードの mp4 です。
- 動画はそのまま提出動画の素材になります。

実行2: 自分で ACT を学習する

```
lerobot-train \
  --policy.type=act \
  --env.type=aloha \
  --env.task=AlohaTransferCube-v0 \
  --dataset.repo_id=lerobot/aloha_sim_transfer_cube_human \
  --output_dir=outputs/train/act_transfer_cube \
  --job_name=act_transfer_cube \
  --policy.device=cuda \
  --policy.push_to_hub=false \
  --steps=100000 \
  --batch_size=8 \
  --eval_freq=20000 \
  --save_freq=20000
```

ここから細かく解説します。

- `--policy.type=act`
- 方策の種類です。`diffusion` / `tdmpc` / `vqbet` / `pi0` / `smolvla` なども指定できます。
- `--dataset.repo_id`
- 人が遠隔操作した50エピソードのデモです。`_scripted` 付きのものは、プログラムで生成したデモです。
- `--policy.push_to_hub=false`
- 新しい版では、Hub にアップロードしない場合にこの指定が必要です。指定しないと、`repo_id` がないというエラーになります。
- `--eval_freq`
- 学習の途中で、シミュレーション上の成功率を自動で測ります。この値を後でグラフにします。
- **学習時間の目安**
- RTX 3060 で 10万ステップに数時間かかります。時間がない場合は `--steps=30000` でもある程度動きます。

学習の途中経過を見る

```
# wandb を使う場合
lerobot-train ... --wandb.enable=true
```

wandb を使わない場合は、ターミナルのログに出る `loss` と、eval のときの `pc_success` を記録しておきます。

学習したモデルの評価

```
lerobot-eval \
  --policy.path=outputs/train/act_transfer_cube/checkpoints/last/pretrained_model \
  --env.type=aloha --env.task=AlohaTransferCube-v0 \
  --eval.n_episodes=50 --eval.batch_size=10 --policy.device=cuda
```

実行3: 比較実験（最終課題の本体）

実験A: ACT と Diffusion Policy を同じタスクで比較

```
# PushT を ACT で学習
lerobot-train --policy.type=act --env.type=pusht --dataset.repo_id=lerobot/pusht \
  --output_dir=outputs/train/act_pusht --policy.device=cuda --policy.push_to_hub=false --steps=100000

# PushT を Diffusion Policy で学習
lerobot-train --policy.type=diffusion --env.type=pusht --dataset.repo_id=lerobot/pusht \
  --output_dir=outputs/train/diffusion_pusht --policy.device=cuda --policy.push_to_hub=false --steps=100000
```

実験B: ACT のチャンクサイズを変える

```
for c in 1 10 50 100; do
  lerobot-train --policy.type=act --policy.chunk_size=$c --policy.n_action_steps=$c \
    --env.type=aloha --env.task=AlohaTransferCube-v0 \
    --dataset.repo_id=lerobot/aloha_sim_transfer_cube_human \
```

```
    --output_dir=outputs/train/act_chunk$c --policy.device=cuda --policy.push_to_hub=false --steps=50000
done
```

- チャンクサイズ1は、毎ステップ1行動だけを予測するので、ACT論文で比較対象にされている「チャンキングなし」に相当します。
- 論文の主張である「チャンキングで成功率が上がる」を、自分で確かめられます。

結果をまとめるスクリプト

```
# summarize.py
import json, glob
for path in sorted(glob.glob("outputs/eval/**/eval_info.json", recursive=True)):
    info = json.load(open(path))
    agg = info.get("aggregated", info.get("overall", {}))
    print(path, "success =", agg.get("pc_success"), "avg_reward =", agg.get("avg_sum_reward"))
```

`eval_info.json` のキー名は版によって違うことがあるので、一度ファイルを開いて確認してください。

工夫ポイント（最終課題として発展させる案）

1. **比較のグラフ化** - 横軸を「学習ステップ」または「デモ数」、縦軸を「成功率」にしたグラフにまとめます。
2. **デモの数を減らす実験** - データセットの一部のエピソードだけで学習します（`--dataset.episodes='[0,1,...,9]'`）。データ効率の検証になります。
3. **失敗例の分析** - 失敗した動画を見て、「掴み損ね」「受け渡しで落とす」などに分類します。考察として高く評価されます。
4. **実機への橋渡し** - A05（SO-101実機）に同じ手順を移します。シミュレーションと実機の差をレポートに書けます。

参考

- LeRobot: https://github.com/huggingface/lerobot
- LeRobot Docs: https://huggingface.co/docs/lerobot
- gym-aloha: https://github.com/huggingface/gym-aloha
- gym-pusht: https://github.com/huggingface/gym-pusht
- Zhao et al., "Learning Fine-Grained Bimanual Manipulation with Low-Cost Hardware" (ACT), RSS 2023
- Chi et al., "Diffusion Policy", RSS 2023

目次へ

A05 SO-101実機でLeRobotの模倣学習をする

環境

- OS: Ubuntu 22.04
- macOS や Windows でも動きますが、ポート名の書き方が違います。
- Python: 3.10
- ライブラリ: LeRobot（`[feetech]` 付き）
- ハードウェア:
- SO-101 リーダーアームとフォロワーアームのセット
  - 2026年9月15日から秋月電子で組み立て済みが販売されています。
  - 部品から自作する場合は https://github.com/TheRobotStudio/SO-ARM100 を参照します。
- USBカメラ1〜2台（正面と手首）
- 学習用のGPU（VRAM 8GB以上）、または Google Colab
- 参考:
- PDFのサンプル: SO-101入門 https://note.com/npaka/n/nfaa9db0834e5
- PDFのサンプル: おもちゃのロボットアームでフィジカルAIを網羅的に勉強したい https://qiita.com/araobp/items/b34a5c64f281f09d3d89
- LeRobot SO-101 ドキュメント: https://huggingface.co/docs/lerobot/so101

概要

人がリーダーアームを手で動かすと、フォロワーアームが同じように動きます（遠隔操作、テレオペレーション）。この動きとカメラ映像を50回分記録し、ACT で学習させると、フォロワーアームが1台で同じ作業をできるようになります。

手順の全体像は次のとおりです。

1. ポートを確認する（`lerobot-find-port`）
2. モータのIDを設定する（`lerobot-setup-motors`）
3. キャリブレーションをする（`lerobot-calibrate`）
4. 遠隔操作で練習する（`lerobot-teleoperate`）
5. デモを記録する（`lerobot-record`、50エピソード）
6. 学習する（`lerobot-train`、ACT）
7. 自律動作させて評価する（`lerobot-record` に `--policy.path` を付ける）

CLIの名前と引数は LeRobot の版によって変わります。迷ったら必ず公式ドキュメント（上記のURL）の最新版に合わせてください。

準備

1. LeRobot のインストール

```
conda create -y -n lerobot python=3.10 && conda activate lerobot
conda install -y ffmpeg -c conda-forge
git clone https://github.com/huggingface/lerobot.git && cd lerobot
pip install -e ".[feetech]"
```

2. ポートを確認する

```
lerobot-find-port
```

表示に従ってUSBケーブルを抜き差しすると、ポート名がわかります（例: フォロワー `/dev/ttyACM0`、リーダー `/dev/ttyACM1`）。Linux では、次のコマンドで権限を与えます。

```
sudo chmod 666 /dev/ttyACM0 /dev/ttyACM1
```

3. モータのIDを設定する（組み立て前、または初回だけ）

```
lerobot-setup-motors --robot.type=so101_follower --robot.port=/dev/ttyACM0
lerobot-setup-motors --teleop.type=so101_leader  --teleop.port=/dev/ttyACM1
```

画面の指示に従って、モータを1つずつつなぎ替えます。組み立て済みの製品では、この作業が不要な場合があります。

4. キャリブレーション

```
lerobot-calibrate --robot.type=so101_follower --robot.port=/dev/ttyACM0 --robot.id=my_follower
lerobot-calibrate --teleop.type=so101_leader  --teleop.port=/dev/ttyACM1 --teleop.id=my_leader
```

- 指示に従って、全関節を中央の位置に合わせてから Enter を押します。
- 次に、各関節を可動範囲いっぱいまで動かします。
- `--robot.id` で付けた名前で、キャリブレーションの結果が保存されます。以後の全コマンドで同じ id を使ってください。

実行

1. 遠隔操作（カメラなしで練習）

```
lerobot-teleoperate \
  --robot.type=so101_follower --robot.port=/dev/ttyACM0 --robot.id=my_follower \
  --teleop.type=so101_leader  --teleop.port=/dev/ttyACM1 --teleop.id=my_leader
```

2. カメラの確認

```
lerobot-find-cameras opencv
```

カメラの番号（`index_or_path`）と解像度を確認します。

3. デモの記録（50エピソード）

```
HF_USER=$(huggingface-cli whoami | head -n 1)

lerobot-record \
  --robot.type=so101_follower --robot.port=/dev/ttyACM0 --robot.id=my_follower \
  --robot.cameras="{ front: {type: opencv, index_or_path: 0, width: 640, height: 480, fps: 30}, wrist: {type: opencv, in
```

```
dex_or_path: 2, width: 640, height: 480, fps: 30}}" \
  --teleop.type=so101_leader --teleop.port=/dev/ttyACM1 --teleop.id=my_leader \
  --display_data=true \
  --dataset.repo_id=${HF_USER}/so101_pick_place \
  --dataset.num_episodes=50 \
  --dataset.episode_time_s=20 \
  --dataset.reset_time_s=10 \
  --dataset.single_task="Pick the red cube and place it in the box"
```

ここから細かく解説します。

- `episode_time_s` / `reset_time_s`
- 1回の作業時間と、物体を元に戻すための時間です。
- `single_task`
- このデータセットの言語指示です。ACT は使いませんが、A06 の SmolVLA では使います。最初から正しく書いておきましょう。
- **キーボード操作（記録中）**
- 右矢印: 今のエピソードを早めに終わらせて次に進む
- 左矢印: 今のエピソードを撮り直す
- Esc: 記録を終了する
- **良いデモのコツ**
- 物体の初期位置を毎回少しずつ変えます（汎化のため）。
- 動きは一定の速さで、迷わずに行います。
- 失敗したデモは撮り直して、データに入れないようにします。

4. 学習（ACT）

```
lerobot-train \
  --dataset.repo_id=${HF_USER}/so101_pick_place \
  --policy.type=act \
  --output_dir=outputs/train/act_so101 \
  --job_name=act_so101 \
  --policy.device=cuda \
  --policy.repo_id=${HF_USER}/act_so101 \
  --steps=60000 --batch_size=8
```

- RTX 3060 で数時間かかります。
- 手元にGPUがない場合は、データセットが Hub にアップロードされているので、Colab で同じコマンドを実行できます。

5. 自律動作と評価

```
lerobot-record \
  --robot.type=so101_follower --robot.port=/dev/ttyACM0 --robot.id=my_follower \
  --robot.cameras="{ front: {type: opencv, index_or_path: 0, width: 640, height: 480, fps: 30}, wrist: {type: opencv, in
dex_or_path: 2, width: 640, height: 480, fps: 30}}" \
  --display_data=true \
  --dataset.repo_id=${HF_USER}/eval_act_so101 \
```

```
--dataset.num_episodes=10 \
--dataset.single_task="Pick the red cube and place it in the box" \
--policy.path=outputs/train/act_so101/checkpoints/last/pretrained_model
```

- `--teleop` を付けずに `--policy.path` を付けると、学習した方策がアームを動かします。
- 評価用データセットの名前は、慣習として `eval_` で始めます。
- 10回中何回成功したかを数えて、成功率を出します。

トラブルシューティング

- **アームが震える、または暴れる**
- キャリブレーションをやり直します。カメラの番号が記録時と同じかも確認します。
- **成功率が低い**
- デモの数を増やす（50 → 100）。
- 照明や背景を記録時と同じにする。
- 手首カメラを追加する。
- `Permission denied: /dev/ttyACM0`
- `sudo usermod -aG dialout $USER` を実行して、再ログインします。

工夫ポイント（最終課題として発展させる案）

1. **デモ数と成功率の関係** - デモ数を 10・25・50 と変えて学習し、成功率を比べます（実機ならではの定量評価）。
2. **初期位置の汎化** - 学習時になかった位置に物体を置いて成功率を測ります。
3. **ACT と SmolVLA の比較** - 同じデータで A06 の SmolVLA を学習させます。言語指示を変えると挙動が変わるかを検証します。
4. **安全対策** - 関節速度の上限や作業領域の制限を入れて、実用性をアピールします。

参考

- LeRobot SO-101: https://huggingface.co/docs/lerobot/so101
- SO-ARM100 / SO-101 ハードウェア: https://github.com/TheRobotStudio/SO-ARM100
- note: SO-101入門 https://note.com/npaka/n/nfaa9db0834e5
- Qiita: おもちゃのロボットアームでフィジカルAIを網羅的に勉強したい https://qiita.com/araobp/items/b34a5c64f281f09d3d89
- 秋月電子 SO-101 販売のニュース: https://innovatopia.jp/robot/robot-news/117891/
- Zhao et al., ACT, RSS 2023

目次へ

A06 SmolVLAで言語指示によるアーム操作

環境

- OS: Ubuntu 22.04
- Python: 3.10
- ライブラリ: LeRobot（`[smolvla]` 付き）
- モデル: `lerobot/smolvla_base`（約4.5億パラメータの VLA = Vision-Language-Action モデル）
- 実行環境（どちらか）:
- 実機: SO-101（A05 で作ったデータセットを使う）
- シミュレーション: LIBERO（LeRobot の `[libero]` 付き）
- GPU:
- 微調整（fine-tuning）: VRAM 16GB以上を推奨。`batch_size` を下げれば 12GB でも可。
- 推論: 8GB程度
- 参考:
- LeRobot SmolVLA ドキュメント: https://huggingface.co/docs/lerobot/smolvla
- 論文: Shukor et al., "SmolVLA: A Vision-Language-Action Model for Affordable and Efficient Robotics", arXiv:2506.01844

概要

SmolVLA は、**画像（複数のカメラ）・ロボットの関節状態・言語指示**を入力として、**連続値の行動列**を出力する小型の VLA です。

- 内部は、小型の視覚言語モデル（SmolVLM）と、Flow Matching で行動を生成する「アクションエキスパート」の組み合わせです。
- `smolvla_base` はコミュニティのデータで事前学習されたベースモデルです。**自分の環境とタスクで微調整しないと、まともに動きません。**
- 公式ドキュメントでは、まず約50エピソードを記録して微調整することが推奨されています。

ACT（A05）との違いは、**言語指示を入力として使う**点です。同じデータセットに「赤いキューブを箱に」「青いキューブを箱に」のような複数のタスクを混ぜて学習させると、指示によって行動を切り替えられるようになります。

準備

```
conda activate lerobot          # A04 / A05 で作った環境
cd lerobot
pip install -e ".[smolvla]"
# シミュレーション（LIBERO）も使う場合
pip install -e ".[libero]"
```

実行1（実機）：言語で切り替えられるデータを集める

A05 と同じ `lerobot-record` を使います。**タスクの文章を変えて**、2〜3種類のタスクを記録します。

```
HF_USER=$(huggingface-cli whoami | head -n 1)
CAMS="{ front: {type: opencv, index_or_path: 0, width: 640, height: 480, fps: 30}, wrist: {type: opencv, index_or_path:
2, width: 640, height: 480, fps: 30}}"

# タスク1: 赤（25エピソード）
lerobot-record --robot.type=so101_follower --robot.port=/dev/ttyACM0 --robot.id=my_follower \
  --robot.cameras="$CAMS" --teleop.type=so101_leader --teleop.port=/dev/ttyACM1 --teleop.id=my_leader \
  --dataset.repo_id=${HF_USER}/so101_color_pick --dataset.num_episodes=25 \
  --dataset.single_task="Pick the red cube and place it in the box"

# タスク2: 青（同じデータセットに追記。25エピソード）
lerobot-record ...（同上）... \
  --dataset.repo_id=${HF_USER}/so101_color_pick --dataset.num_episodes=25 --resume=true \
  --dataset.single_task="Pick the blue cube and place it in the box"
```

- 赤と青のキューブを**両方とも机に置いた状態で**記録するのがポイントです。
- 片方しか置かないと、モデルは言語を見なくても「見えている方を取る」だけで正解できてしまいます。そうなると、言語理解の検証になりません。
- `--resume=true` は既存のデータセットに追記するための指定です。版によって名前が違う可能性があるので、`lerobot-record --help` で確認してください。

実行2: 微調整

```
lerobot-train \
  --policy.path=lerobot/smolvla_base \
  --dataset.repo_id=${HF_USER}/so101_color_pick \
  --batch_size=32 \
  --steps=20000 \
  --output_dir=outputs/train/smolvla_color \
  --job_name=smolvla_color \
  --policy.device=cuda \
  --policy.push_to_hub=false
```

- **`--policy.path=lerobot/smolvla_base`**
- `--policy.type` ではなく `--policy.path` で、**事前学習の重みから**始めます。ここが ACT との違いです。
- **学習時間**
- 公式の目安では、A100 で 2万ステップに約4時間かかります。RTX 3060 では `--batch_size=8` に下げて半日程度を見込んでください。
- Colab の A100 / L4 を使うのが現実的です。

実行3: 言語を変えて推論する

```
# 赤を指示
lerobot-record --robot.type=so101_follower --robot.port=/dev/ttyACM0 --robot.id=my_follower \
  --robot.cameras="$CAMS" \
  --dataset.repo_id=${HF_USER}/eval_smolvla_red --dataset.num_episodes=10 \
  --dataset.single_task="Pick the red cube and place it in the box" \
```

```
  --policy.path=outputs/train/smolvla_color/checkpoints/last/pretrained_model

# 青を指示（同じ配置で）
lerobot-record ...（同上）... --dataset.repo_id=${HF_USER}/eval_smolvla_blue \
  --dataset.single_task="Pick the blue cube and place it in the box"
```

実行4（シミュレーション）: LIBERO で評価する

実機がない場合は、LeRobot に統合されている LIBERO 環境で評価します。

```
lerobot-eval \
  --policy.path=<LIBEROで微調整済みのSmolVLAのrepo_id> \
  --env.type=libero \
  --env.task=libero_object \
  --eval.batch_size=1 \
  --eval.n_episodes=10
```

- LIBERO で微調整済みの SmolVLA のチェックポイントは、Hugging Face Hub で `smolvla libero` を検索して、最新のものを選んでください。名前が変わることがあります。
- LIBERO には4つのタスク群（suite）があります。
- `libero_spatial`：配置の違い
- `libero_object`：物体の違い
- `libero_goal`：目標の違い
- `libero_10`：長いタスク

Pythonから直接推論する（理解を深めるため）

```
import torch
from lerobot.policies.smolvla.modeling_smolvla import SmolVLAPolicy

policy = SmolVLAPolicy.from_pretrained("outputs/train/smolvla_color/checkpoints/last/pretrained_model")
policy.eval().to("cuda")

batch = {
    "observation.images.front": front_img,   # (1, 3, H, W) float 0-1
    "observation.images.wrist": wrist_img,
    "observation.state": state,              # (1, 6) 関節角
    "task": ["Pick the blue cube and place it in the box"],
}
with torch.no_grad():
    action = policy.select_action(batch)     # (1, 6) 次の関節目標
```

- import のパスは版によって `lerobot.common.policies...` の場合があります。
- 入力キーの名前は、データセットの `meta/info.json` の features と一致させます。

工夫ポイント（最終課題として発展させる案）

1. **言語への感度の検証** - 次の3種類の指示で成功率を比べます。

    - 学習時の文そのまま
    - 言い換え（"Put the blue block into the container"）
    - 学習時にない色（"green"）
    - VLA の汎化の限界を示すと、良い考察になります。
2. **ACT との比較** - 同じデータ量で A05 の ACT と比べます。言語の条件付けがないと、2つのタスクを区別できないことを示します。
3. **音声入力** - Whisper で音声を文字に起こし、`task` に入れます。「話しかけると動くアーム」として表現力の評価が上がります。

参考

- SmolVLA Docs: https://huggingface.co/docs/lerobot/smolvla
- SmolVLA 論文: https://arxiv.org/abs/2506.01844
- LeRobot: https://github.com/huggingface/lerobot
- LIBERO: https://github.com/Lifelong-Robot-Learning/LIBERO

目次へ

A07 Gemini Robotics-ER 2 で机の上を片付ける（Genesis + Franka）

環境

- OS: Ubuntu 22.04（Colab 可）
- Python: 3.10
- シミュレータ: Genesis
- ロボット: Franka Emika Panda（Genesis に付属するMJCF）
- AI: Gemini Robotics-ER 2（`gemini-robotics-er-2-preview`）。物体の指示座標（pointing）とタスク計画に使い、推論のみです。
- SDK: `google-genai`
- APIキー: Google AI Studio で取得します（講座からは提供されません）。
- 参考:
- Gemini Robotics-ER ドキュメント（日本語）: https://ai.google.dev/gemini-api/docs/generate-content/robotics-overview?hl=ja
- Gemini API Cookbook（Robotics-ER の quickstart）: https://github.com/google-gemini/cookbook
- PDFのサンプル: Gemini Robotics-ER 1.6を試す https://note.com/npaka/n/n93116b196748

重要: 公式のリリースノートによると、`gemini-robotics-er-1.6-preview` は 2026年8月31日に提供を終了し、ER 2 への移行が案内されています。PDFのサンプル記事のコードを使う場合は、モデル名を `gemini-robotics-er-2-preview` に書き換えてください。

概要

机の上に散らばった色付きキューブを、Genesis のカメラで撮影し、画像を Gemini Robotics-ER に送ります。Gemini には次の2つを答えさせます。

1. **Pointing**: 各物体が画像のどこにあるか（`[y, x]`、0〜1000 に正規化された座標）。
2. **Planning**: どの順番で、どこ（左右のトレイ）に片付けるか（JSON）。

返ってきた点の画素を**セグメンテーション画像**で物体の ID に変換し、その物体の3D位置に Franka を動かしてピック＆プレースします。

処理の流れは次のとおりです。

1. Genesis のカメラで rgb 画像とセグメンテーション画像を撮る。
2. rgb 画像を Gemini Robotics-ER 2 に送り、点と片付け計画（JSON）を受け取る。
3. 点の画素位置をセグメンテーション画像で引き、どの物体かを特定して、その3D位置を取る。
4. `franka_api.pick()` / `place()` でアームを動かす。
5. これを全物体について繰り返し、最後に動画を mp4 で書き出す。

この記事では、**B05・B06・B07 でも使う共通部品** `franka_api.py` を最初に作ります。

準備

```
conda create -n genesis python=3.10 -y && conda activate genesis
pip install torch --index-url https://download.pytorch.org/whl/cu121
pip install genesis-world google-genai pillow numpy
export GEMINI_API_KEY="（AI Studioで取得したキー）"
```

ソースコード

共通部品: franka_api.py

Genesis 公式チュートリアル（IK & Motion Planning）の書き方をもとに、「物体を掴む」「置く」を関数にまとめたものです。

```
# franka_api.py
import numpy as np
import genesis as gs


def to_np(x):
    return x.cpu().numpy() if hasattr(x, "cpu") else np.asarray(x)


class FrankaAPI:
    """Genesis 上の Franka を「pick / place」で操作する簡易API"""

    GRASP_OFFSET = 0.11   # hand リンクと指先の距離（物体中心からの高さ）
    LIFT = 0.25           # 物体の上空に待機する高さ

    def __init__(self, scene, franka, cam=None):
        self.scene, self.franka, self.cam = scene, franka, cam
        self.motors = np.arange(7)
        self.fingers = np.arange(7, 9)
        self.ee = franka.get_link("hand")
        self.down = np.array([0, 1, 0, 0])   # 手先を真下に向ける姿勢 (w, x, y, z)
        franka.set_dofs_kp(np.array([4500, 4500, 3500, 3500, 2000, 2000, 2000, 100, 100]))
        franka.set_dofs_kv(np.array([450, 450, 350, 350, 200, 200, 200, 10, 10]))
        franka.set_dofs_force_range(
            np.array([-87, -87, -87, -87, -12, -12, -12, -100, -100]),
            np.array([87, 87, 87, 87, 12, 12, 12, 100, 100]))
        self.holding = None

    def step(self, n=1):
        for _ in range(n):
            self.scene.step()
            if self.cam is not None:
                self.cam.render()

    def move_to(self, pos, steps=120):
        """手先を pos (x, y, z) へ。関節空間で直線補間して動かす"""
```

```
        q_goal = to_np(self.franka.inverse_kinematics(
            link=self.ee, pos=np.array(pos, dtype=float), quat=self.down))
        q_now = to_np(self.franka.get_dofs_position())
        for i in range(1, steps + 1):
            q = q_now + (q_goal - q_now) * i / steps
            self.franka.control_dofs_position(q[:7], self.motors)
            self.step()
        self.step(30)   # 収束待ち

    def open_gripper(self):
        self.franka.control_dofs_position(np.array([0.04, 0.04]), self.fingers)
        self.step(60)

    def close_gripper(self):
        self.franka.control_dofs_force(np.array([-0.5, -0.5]), self.fingers)
        self.step(80)

    def pick(self, obj_pos):
        x, y, z = obj_pos
        self.open_gripper()
        self.move_to((x, y, z + self.LIFT))
        self.move_to((x, y, z + self.GRASP_OFFSET), steps=80)
        self.close_gripper()
        self.move_to((x, y, z + self.LIFT), steps=80)

    def place(self, target_pos):
        x, y, z = target_pos
        self.move_to((x, y, z + self.LIFT))
        self.move_to((x, y, z + self.GRASP_OFFSET + 0.02), steps=80)
        self.open_gripper()
        self.move_to((x, y, z + self.LIFT), steps=80)

    def home(self):
        self.move_to((0.45, 0.0, 0.45))


def make_scene(objects, show_viewer=False, cam_pos=(1.3, 0.0, 1.0), cam_lookat=(0.5, 0.0, 0.0)):
    """objects: {"red_cube": ((x, y, z), (r, g, b)), ...}"""
    gs.init(backend=gs.gpu)
    scene = gs.Scene(
        sim_options=gs.options.SimOptions(dt=0.01),
        vis_options=gs.options.VisOptions(segmentation_level="entity"),
        show_viewer=show_viewer,
    )
    scene.add_entity(gs.morphs.Plane())
    entities = {}
    for name, (pos, color) in objects.items():
        entities[name] = scene.add_entity(
            gs.morphs.Box(size=(0.04, 0.04, 0.04), pos=pos),
```

```
            surface=gs.surfaces.Default(color=color))
    franka = scene.add_entity(gs.morphs.MJCF(file="xml/franka_emika_panda/panda.xml"))
    cam = scene.add_camera(res=(640, 480), pos=cam_pos, lookat=cam_lookat, fov=45, GUI=False)
    scene.build()
    return scene, franka, entities, cam
```

ここから細かく解説します。

- `set_dofs_kp/kv/force_range`
- 公式チュートリアルと同じ値です。これを設定しないと、PD制御のゲインが弱く、アームが垂れ下がります。
- `move_to()`
- IK で目標の関節角を求め、今の関節角から**直線補間**で少しずつ指令します。
- 公式チュートリアルの `plan_path()`（OMPL を使った経路計画）は、環境によってインストールが面倒です。今回は障害物がない机の上なので、補間で十分です。障害物を避けたい場合は `plan_path` に置き換えます。
- `close_gripper()`
- 位置ではなく**力**で指を閉じます（公式と同じ方式）。物体の大きさに関係なく掴めます。
- `segmentation_level="entity"`
- セグメンテーション画像の値を「エンティティ単位」にします。ピクセルの値から、どの物体が写っているかを引けるようになります。

メイン: tidy_with_gemini.py

```
import io, json, re
import numpy as np
from PIL import Image
from google import genai
from google.genai import types
from franka_api import FrankaAPI, make_scene, to_np

MODEL = "gemini-robotics-er-2-preview"
client = genai.Client()   # 環境変数 GEMINI_API_KEY を使う

OBJECTS = {
    "red_cube":    ((0.55,  0.15, 0.02), (1.0, 0.1, 0.1)),
    "blue_cube":   ((0.60, -0.10, 0.02), (0.1, 0.2, 1.0)),
    "green_cube":  ((0.45,  0.05, 0.02), (0.1, 0.8, 0.1)),
    "yellow_cube": ((0.50, -0.20, 0.02), (1.0, 0.9, 0.1)),
}
TRAYS = {"left": (0.30, 0.35, 0.02), "right": (0.30, -0.35, 0.02)}



def ask(image, prompt):
    buf = io.BytesIO()
    Image.fromarray(image).save(buf, format="PNG")
    resp = client.models.generate_content(
        model=MODEL,
        contents=[types.Part.from_bytes(data=buf.getvalue(), mime_type="image/png"), prompt],
        config=types.GenerateContentConfig(temperature=0.5),
    )
```

```
    text = re.sub(r"^```(json)?|```$", "", resp.text.strip(), flags=re.M).strip()
    return json.loads(text)


POINT_PROMPT = """
画像内の机の上にあるすべての立方体を指してください。
次のJSON形式だけで答えてください。点は [y, x] の順で、0-1000 に正規化した値です。
[{"point": [y, x], "label": "<色>の立方体"}]
"""

PLAN_PROMPT = """
あなたはロボットアームのタスクプランナーです。机の上を片付けます。
ルール: 暖色（赤・黄・オレンジ）は left トレイ、寒色（青・緑）は right トレイへ。
アームの手前（画像の下側）にある物体から順に片付けてください。
次のJSON形式だけで答えてください。
[{"label": "<色>の立方体", "point": [y, x], "tray": "left" または "right", "reason": "<短い理由>"}]
"""


def main():
    scene, franka, ents, cam = make_scene(OBJECTS)
    api = FrankaAPI(scene, franka, cam)
    cam.start_recording()
    api.home()

    rgb, _, seg, _ = cam.render(rgb=True, segmentation=True)
    rgb = np.asarray(rgb, dtype=np.uint8)
    Image.fromarray(rgb).save("scene.png")

    # 1) pointing: 物体の位置を確認（レポート用）
    points = ask(rgb, POINT_PROMPT)
    print("points:", points)

    # 2) planning: 片付け順序とトレイの割り当て
    plan = ask(rgb, PLAN_PROMPT)
    print(json.dumps(plan, ensure_ascii=False, indent=2))

    # セグメンテーションIDからエンティティを引く表
    id2name = {ent.idx: name for name, ent in ents.items()}
    h, w = seg.shape[:2]

    for step in plan:
        y, x = step["point"]
        u, v = int(x / 1000 * w), int(y / 1000 * h)
        seg_id = int(np.asarray(seg)[v, u])
        name = id2name.get(seg_id)
        if name is None:
            print(f"点 {step['point']} は物体に当たっていません（seg={seg_id}）。スキップ")
            continue
```

```
        pos = to_np(ents[name].get_pos())
        print(f"{step['label']} -> {name} at {pos.round(3)} -> tray {step['tray']} ({step['reason']})")
        api.pick(pos)
        api.place(TRAYS[step["tray"]])

    api.home()
    cam.stop_recording(save_to_filename="tidy_gemini.mp4", fps=60)


if __name__ == "__main__":
    main()
```

ここから細かく解説します。

- **座標の形式**
- Gemini Robotics-ER は点を `[y, x]` **の順、0〜1000 に正規化**して返します。
- x と y を逆にするミスが一番多いので注意してください。
- **画素 → 物体の変換**
- 画像上の点を3D座標に直すには、本来はカメラの内部パラメータと深度が必要です。
- シミュレーションではセグメンテーション画像が使えるので、**点の画素がどのエンティティに当たっているか**を引くだけで済みます。
- Gemini が点を物体の端に打つとハズレになることがあります。そのため、ハズレのときはスキップするようにしています。工夫ポイントの1で改善できます。
- `ent.idx` **とセグメンテーションの値**
- 対応関係は Genesis の版で変わることがあります。うまく引けない場合は、`print(np.unique(seg))` と `{n: e.idx for n, e in ents.items()}` を表示して比べてください。
- 値が1つずれている場合は、`seg_id - 1` のように調整します。
- `ask()`
- 応答がコードブロック（`json ...`）で返ってくることがあるので、正規表現で取り除いてから JSON として読み込みます。

実行

```
python tidy_with_gemini.py
```

実行結果の見方

- コンソールに、Gemini が返した計画（順番・トレイ・理由）が表示されます。そのままレポートに貼れます。
- `scene.png` は Gemini に送った画像です。返ってきた点を描き込むと、Pointing がうまくいったかを図で示せます。
- `tidy_gemini.mp4` は提出動画の素材です。

工夫ポイント（最終課題として発展させる案）

1. **点が外れたときの頑健化** - 点の周りの 5×5 画素で、一番多いセグメンテーションIDを採用します。
2. **閉ループ化（再計画）** - 1つ片付けるごとに画像を撮り直し、「残っている物体」を Gemini に再計画させます。物体が落ちたときなどにも対応できます。
3. **曖昧な指示** - 「食べ物っぽい色のものだけ片付けて」のような常識が必要な指示を与え、VLM ならではの推論を見せます。

4. **軌道の出力** - ER 2 に「障害物を避ける軌道を点列で」出させ、`move_to` を順番にたどります。
5. **比較** - ルールベース（色を閾値で判定）と Gemini の計画を、成功率と柔軟性で比べます。

参考

- Gemini Robotics-ER ドキュメント: https://ai.google.dev/gemini-api/docs/generate-content/robotics-overview?hl=ja
- Gemini Robotics-ER 2 preview: https://ai.google.dev/gemini-api/docs/models/gemini-robotics-er-2-preview
- Gemini API Cookbook: https://github.com/google-gemini/cookbook
- Genesis IK & Motion Planning チュートリアル: https://genesis-world.readthedocs.io/
- Gemini Robotics Team, "Gemini Robotics: Bringing AI into the Physical World", 2025

目次へ

A08 ROSA（NASA JPL）で自然言語からROS2ロボットを操作する

環境

- OS: Ubuntu 22.04
- ROS: ROS 2 Humble
- シミュレータ: Gazebo Classic 11（TurtleBot3 Waffle Pi。A01 の環境）
- AI:
- ROSA（`jpl-rosa`、LangChain ベースのROSエージェント）
- LLM: Ollama のローカルLLM（例: `qwen2.5:7b`）。OpenAI などの API キーがあれば、そちらでも動きます。
- 参考リポジトリ: https://github.com/nasa-jpl/rosa
- 論文: "Enabling Novel Mission Operations and Interactions with ROSA: The Robot Operating System Agent", arXiv:2410.06472
- PDFのサンプル: ChatGPT APIによる自然言語ロボット操作 https://qiita.com/porizou1/items/a085d20e936946330540

概要

ROSA は、自然言語の質問や命令を ROS の操作に変換する AI エージェントです。

- 標準で「トピック一覧を見る」「ノード情報を見る」「パラメータを読む」などのツールを持っています。
- LLM が**どのツールをどの引数で呼ぶか**を自分で決めます（Function Calling / Tool Use）。

この記事では、ROSA に **TurtleBot3 を動かす自作ツール**（前進・回転・現在位置の取得）を追加し、「1m前に進んでから90度左を向いて、今の位置を教えて」のような指示で、Gazebo の中のロボットを操作します。

処理の流れは次のとおりです。

1. ユーザーが自然言語で指示する。
2. ROSA（LLMエージェント）が、どのツールを使うかを決める。 - 標準ツール: トピック一覧、ノード情報など - 自作ツール: `move_forward(m)`、`rotate(deg)`、`get_pose()`
3. 自作ツールが `/cmd_vel` と `/odom` を通じて TurtleBot3（Gazebo）を動かす。

APIについて: PDFにあるとおり、ChatGPT などの API は講座から提供されません。この記事では、無料で使える Ollama のローカルLLMで作ります。ツール呼び出しに対応したモデル（qwen2.5、llama3.1 など）を選んでください。

ライセンス: ROSA には JPL 独自のライセンス条項があります。課題として使う分には問題になりにくいですが、リポジトリの LICENSE を一読しておきましょう。

準備

```
# A01 の TurtleBot3 環境がある前提
pip install jpl-rosa langchain-ollama

# Ollama（ローカルLLM）
curl -fsSL https://ollama.com/install.sh | sh
ollama pull qwen2.5:7b
```

- VRAM 8GB程度なら `qwen2.5:7b` が快適に動きます。
- VRAM がない場合は、CPU で `qwen2.5:3b` を使います（遅いですが動きます）。

ソースコード

rosa_turtlebot.py

```
import math
import time

import rclpy
from geometry_msgs.msg import Twist
from nav_msgs.msg import Odometry
from langchain.agents import tool
from langchain_ollama import ChatOllama
from rosa import ROSA

# ---- ROS 2 の準備（ツールから使う） ----
rclpy.init()
node = rclpy.create_node("rosa_tools")
cmd_pub = node.create_publisher(Twist, "/cmd_vel", 10)
latest_odom = {}


def on_odom(msg):
    latest_odom["msg"] = msg


node.create_subscription(Odometry, "/odom", on_odom, 10)


def yaw_from_quat(q):
    return math.atan2(2 * (q.w * q.z + q.x * q.y), 1 - 2 * (q.y * q.y + q.z * q.z))


def publish_for(linear, angular, duration):
    cmd = Twist()
    cmd.linear.x, cmd.angular.z = linear, angular
    end = time.time() + duration
    while time.time() < end:
        cmd_pub.publish(cmd)
        rclpy.spin_once(node, timeout_sec=0.05)
    cmd_pub.publish(Twist())   # 停止


# ---- 自作ツール（docstring が LLM への説明になる） ----
@tool
def move_forward(distance_m: float) -> str:
    """ロボットを distance_m メートル前進させる。負の値なら後退する。1回の上限は2m。"""
```

```
    distance_m = max(-2.0, min(2.0, distance_m))
    speed = 0.15
    publish_for(math.copysign(speed, distance_m), 0.0, abs(distance_m) / speed)
    return f"{distance_m:.2f} m 移動しました。"


@tool
def rotate(angle_deg: float) -> str:
    """ロボットをその場で angle_deg 度回転させる。正なら左回り（反時計回り）、負なら右回り。"""
    w = 0.5
    publish_for(0.0, math.copysign(w, angle_deg), abs(math.radians(angle_deg)) / w)
    return f"{angle_deg:.0f} 度回転しました。"


@tool
def get_pose() -> str:
    """ロボットの現在位置 (x, y) [m] と向き yaw [deg] をオドメトリから返す。"""
    for _ in range(20):
        rclpy.spin_once(node, timeout_sec=0.05)
    if "msg" not in latest_odom:
        return "オドメトリを取得できませんでした。"
    p = latest_odom["msg"].pose.pose
    yaw = math.degrees(yaw_from_quat(p.orientation))
    return f"x={p.position.x:.2f} m, y={p.position.y:.2f} m, yaw={yaw:.0f} deg"


# ---- ROSA エージェント ----
llm = ChatOllama(model="qwen2.5:7b", temperature=0)
agent = ROSA(
    ros_version=2,
    llm=llm,
    tools=[move_forward, rotate, get_pose],
)

if __name__ == "__main__":
    print("指示を入力してください（exit で終了）")
    while True:
        q = input("> ")
        if q.strip() in ("exit", "quit"):
            break
        print(agent.invoke(q))
    cmd_pub.publish(Twist())
    node.destroy_node()
    rclpy.shutdown()
```

ここから細かく解説します。

- **`@tool` の docstring**
- LLM はこの説明文を読んで、どのツールを使うか、どんな引数を渡すかを決めます。

- 単位（m、deg）・符号の意味・上限を書いておくと、誤作動が大きく減ります。ここがプロンプトエンジニアリングの肝です。
- **安全のための上限**
- `move_forward` の中で ±2m に制限しています。LLM が「100m進む」と解釈しても暴走しないためです。レポートの「実用性」で書けるポイントになります。
- **時間ベースの移動**
- 速度×時間で距離を出すので、多少の誤差が出ます。工夫ポイントの1で、オドメトリを見て止める方式に改善できます。
- `ROSA(ros_version=2, ...)`
- ROS 2 の CLI（`ros2 topic list` など）をラップした標準ツールも自動で使えるようになります。そのため、「今どんなトピックがある？」と聞くことができます。
- **import のパス**
- `from langchain.agents import tool` は ROSA の README にならった書き方です。LangChain の版によっては、`from langchain_core.tools import tool` に変える必要があります。

実行

端末1: シミュレータを起動

```
export TURTLEBOT3_MODEL=waffle_pi
ros2 launch turtlebot3_gazebo turtlebot3_world.launch.py
```

端末2: Ollama を起動（サービスとして起動済みなら不要）

```
ollama serve
```

端末3: エージェントを起動

```
source ~/turtlebot3_ws/install/setup.bash
python3 rosa_turtlebot.py
```

入力例

```
> 今どんなトピックが出ている？
> 1m前に進んでから左に90度向いて、最後に今の位置を教えて
> 一辺0.5mの正方形を描くように動いて
```

実行結果の見方

- `verbose=True` を ROSA に渡すと、LLM がどのツールをどの引数で呼んだか（思考の過程）が表示されます。
- 「正方形を描いて」の場合は、`move_forward(0.5)` → `rotate(90)` が4回繰り返し呼ばれれば成功です。
- この呼び出しの記録をスクリーンショットで動画に入れると、「AIがどう判断したか」が伝わります。

工夫ポイント（最終課題として発展させる案）

1. **閉ループ制御** - `move_forward` を「オドメトリで移動距離を測って止める」方式にし、時間ベースとの誤差を比べます。

2. **Nav2 との連携** - `go_to(place_name)` ツールを作り、「キッチン」のような地名から座標の辞書を引いて、Nav2 の `NavigateToPose` アクションに送ります。部屋の中を言葉で移動できるようになります。
3. **カメラとの連携** - A01 の YOLO 検出結果を返す `what_do_you_see()` ツールを追加し、「人を探して近づいて」のような指示に対応させます。
4. **LLM の比較** - 3B / 7B / 14B のモデルで、同じ10個の指示に対する正しいツール呼び出しの割合を比べます。
5. **音声入力** - Whisper（`openai-whisper` または `faster-whisper`）で音声を文字にして `agent.invoke` に渡します。

参考

- ROSA: https://github.com/nasa-jpl/rosa
- ROSA 論文: https://arxiv.org/abs/2410.06472
- jpl-rosa (PyPI): https://pypi.org/project/jpl-rosa
- Ollama: https://ollama.com/
- LangChain Ollama: https://python.langchain.com/docs/integrations/chat/ollama/
- Qiita: ChatGPT APIによる自然言語ロボット操作 https://qiita.com/porizou1/items/a085d20e936946330540

目次へ

A09 Ultralytics YOLOを自作データで学習させる

環境

- OS: Ubuntu 22.04（学習だけなら Windows や Colab でも可）
- ROS: ROS 2 Humble（画像の収集に使う）
- シミュレータ: Gazebo Classic 11（TurtleBot3 Waffle Pi）
- AI: Ultralytics YOLO11（学習あり）
- アノテーションツール: Label Studio（または CVAT、Roboflow）
- 参考リポジトリ: https://github.com/ultralytics/ultralytics

概要

COCO で学習済みの YOLO は80クラスしか知りません。そのため、「Gazebo に置いた自作の目印」や「特定の色の箱」は検出できません。この記事では、次の手順で**自分専用の検出器**を作ります。

1. Gazebo の中でロボットを動かしながら、カメラ画像を保存する。
2. 画像に枠を付ける（アノテーション）。
3. YOLO11n を転移学習させる。
4. mAP（検出精度の指標）で評価する。
5. A01 の追従ノードに組み込む。

この記事は単体でも課題になりますが、**A01・A02・A10 の認識部分を強化する部品**としても使えます。「学習あり」は加点要素の「技術力」に効きます。

別解（学習なし）: 物体の名前を文字で指定できる YOLO-World（A02 で使用）を使えば、学習せずに未知の物体を検出できます。学習した YOLO と YOLO-World を精度と速度で比べるのも、良い考察になります。

準備

```
pip install ultralytics "numpy<2"
pip install label-studio   # アノテーション用（別の venv に入れると依存関係が衝突しにくい）
```

手順1: Gazebo で画像を集める

image_saver.py

```
import os, time
import cv2
import rclpy
from rclpy.node import Node
from sensor_msgs.msg import Image
from cv_bridge import CvBridge


class Saver(Node):
    def __init__(self, out_dir="dataset_raw", interval=0.5):
```

```
        super().__init__("image_saver")
        os.makedirs(out_dir, exist_ok=True)
        self.out_dir, self.interval = out_dir, interval
        self.last = 0.0
        self.bridge = CvBridge()
        self.create_subscription(Image, "/camera/image_raw", self.cb, 10)
        self.count = 0

    def cb(self, msg):
        if time.time() - self.last < self.interval:
            return
        self.last = time.time()
        img = self.bridge.imgmsg_to_cv2(msg, "bgr8")
        cv2.imwrite(f"{self.out_dir}/img_{self.count:05d}.jpg", img)
        self.count += 1
        self.get_logger().info(f"saved {self.count}", throttle_duration_sec=2.0)


def main():
    rclpy.init()
    rclpy.spin(Saver())


if __name__ == "__main__":
    main()
```

実行

```
# 端末1
ros2 launch turtlebot3_gazebo turtlebot3_world.launch.py
# 端末2: キーボードでロボットを動かす
ros2 run turtlebot3_teleop teleop_keyboard
# 端末3: 0.5秒ごとに画像を保存
python3 image_saver.py
```

集め方のコツ

- 対象の物体（例: Gazebo に置いた色付きのコーンや自作モデル）を、いろいろな**距離・角度・背景**から撮ります。
- 200〜500枚を目安にします。
- 対象が写っていない画像も1〜2割混ぜます（誤検出を減らすため）。
- Gazebo の Insert タブで照明（Point light）を追加・移動して明るさを変えると、汎化しやすくなります。

手順2: アノテーション

```
label-studio start
```

1. ブラウザで http://localhost:8080 を開き、プロジェクトを作ります。
2. Labeling Setup で「Object Detection with Bounding Boxes」を選び、クラス名（例: `cone`, `marker`）を登録します。

3. `dataset_raw/` の画像をインポートし、枠を付けます。
4. Export で **YOLO** 形式を選んでダウンロードします。

データの配置

```
dataset/
├── data.yaml
├── images/
│   ├── train/    （8割）
│   └── val/      （2割）
└── labels/
    ├── train/
    └── val/
```

ラベルファイル（`.txt`）は1行が1物体で、`クラスID 中心x 中心y 幅 高さ`（0〜1 に正規化）の形式です。

分割スクリプト split.py

```
import random, shutil
from pathlib import Path

src_img, src_lbl = Path("export/images"), Path("export/labels")
random.seed(0)
imgs = sorted(src_img.glob("*.jpg"))
random.shuffle(imgs)
n_val = int(len(imgs) * 0.2)
for i, img in enumerate(imgs):
    split = "val" if i < n_val else "train"
    for d in ("images", "labels"):
        Path(f"dataset/{d}/{split}").mkdir(parents=True, exist_ok=True)
    shutil.copy(img, f"dataset/images/{split}/{img.name}")
    lbl = src_lbl / (img.stem + ".txt")
    if lbl.exists():
        shutil.copy(lbl, f"dataset/labels/{split}/{lbl.name}")
```

dataset/data.yaml

```
path: /home/<user>/dataset
train: images/train
val: images/val
names:
  0: cone
  1: marker
```

手順3: 学習

```
yolo detect train data=dataset/data.yaml model=yolo11n.pt epochs=100 imgsz=640 batch=16 device=0
```

- `model=yolo11n.pt`
- COCO で学習済みの重みから始めます（転移学習）。少ないデータでも精度が出ます。
- **GPU がない場合**
- `device=cpu` でも動きますが遅いです。Colab を使うと速いです。
- **学習の結果**
- `runs/detect/train/` に保存されます。
- `weights/best.pt` が一番精度の良かった重みです。
- `results.png` が学習曲線です。

手順4: 評価

```
yolo detect val model=runs/detect/train/weights/best.pt data=dataset/data.yaml
```

- **mAP50**: IoU 0.5 以上で正解とみなしたときの平均精度です。
- **mAP50-95**: より厳しい基準での平均精度です。
- `confusion_matrix.png` で、どのクラス同士を間違えやすいかがわかります。
- これらの図をレポートに貼ると説得力が出ます。

推論の確認

```
yolo detect predict model=runs/detect/train/weights/best.pt source=dataset/images/val save=True
```

手順5: ROS ノードへの組み込み

A01 の `follower_node` は、モデルをパラメータで切り替えられるように作ってあります。

```
ros2 launch yolo_follower follower.launch.py target:=cone
# モデルの指定（launch に model 引数を足すか、直接ノードを起動する）
ros2 run yolo_follower follower_node --ros-args -p model:=/home/<user>/runs/detect/train/weights/best.pt -p target:=cone
```

工夫ポイント（最終課題として発展させる案）

1. **シミュレーションだけで集めた画像で、実写でも動くか（Sim-to-Real）** - 実機やWebカメラの画像で評価します。
2. **データ拡張の効果** - `hsv_h`、`degrees`、`mosaic` などの拡張パラメータを変えて、mAP を比べます。
3. **YOLO-World との比較** - 学習なしの YOLO-World と、学習した YOLO11n を、mAP と推論速度（FPS）で比べます。
4. **自動アノテーション** - Gazebo では物体の真の位置がわかります。カメラのパラメータで画像に投影すれば、**自動で枠を付けられます**。人手で枠を付ける作業を省けるので、技術力のアピールになります。

参考

- Ultralytics: https://github.com/ultralytics/ultralytics
- Ultralytics Docs（Train / Val / Predict）: https://docs.ultralytics.com/modes/
- Label Studio: https://labelstud.io/
- YOLO-World: https://docs.ultralytics.com/models/yolo-world/

目次へ

A10 停止標識と歩行者を認識して止まる自動走行ロボット

環境

- OS: Ubuntu 22.04
- ROS: ROS 2 Humble
- シミュレータ: Gazebo Classic 11
- ロボット: TurtleBot3 Waffle Pi（カメラと LiDAR 付き）
- AI: Ultralytics YOLO11n（COCO で学習済み。推論のみ）
- `stop sign`（停止標識）と `person`（人）はどちらも COCO のクラスなので、学習なしで検出できます。
- 参考リポジトリ:
- https://github.com/Tinker-Twins/YOLO-ROS-2 （Darknet YOLO で TurtleBot3 を停止標識に従わせるデモ）
- https://github.com/ROBOTIS-GIT/turtlebot3_applications

概要

TurtleBot3 が直進しながら、次の3つの交通ルールを守ります。

1. **停止標識**: 近づいたら3秒停止し、その後は発進する（同じ標識で何度も止まらないよう、5秒間は無視する）。
2. **歩行者**: 前方に人がいる間は止まり続ける（歩行者優先）。
3. **障害物**: LiDAR で前方0.35m以内に物があれば緊急停止する（AIが見逃したときの保険）。

状態機械は次のように遷移します。

- DRIVE（直進）
- 停止標識が大きく見えた → STOP_SIGN（3秒停止）
- 人が見えた → YIELD（人がいなくなるまで停止）
- LiDAR で0.35m以内に物がある → EMERGENCY（緊急停止）
- STOP_SIGN は、3秒経ったら DRIVE に戻る（同じ標識は5秒間無視する）。
- YIELD は、人が見えなくなったら DRIVE に戻る。
- EMERGENCY は、障害物がなくなったら DRIVE に戻る。

今回のポイントは、**AI（カメラ）とルール（LiDAR）を組み合わせて安全性を確保する**ことです。自動運転の基本的な考え方で、評価基準の「実用性」に直結します。

準備

A01 の手順で TurtleBot3 シミュレーションと ultralytics をインストールしておきます。Gazebo のモデルデータベースに停止標識（`Stop Sign`）と人（`Person Standing`）が必要なので、次のコマンドで取得します。

```
cd ~/.gazebo && git clone https://github.com/osrf/gazebo_models.git models_tmp
cp -r models_tmp/stop_sign models_tmp/person_standing ~/.gazebo/models/
```

ソースコード

traffic_rules/traffic_rules/traffic_node.py

```
import time
import numpy as np
import rclpy
from rclpy.node import Node
from rclpy.qos import qos_profile_sensor_data
from sensor_msgs.msg import Image, LaserScan
from geometry_msgs.msg import Twist
from cv_bridge import CvBridge
from ultralytics import YOLO



class TrafficRules(Node):
    def __init__(self):
        super().__init__("traffic_rules")
        self.declare_parameter("speed", 0.15)
        self.declare_parameter("stop_sign_area", 0.02)   # 画面の何割で「近い」とみなすか
        self.declare_parameter("person_area", 0.03)
        self.declare_parameter("emergency_dist", 0.35)
        self.speed = self.get_parameter("speed").value
        self.stop_area = self.get_parameter("stop_sign_area").value
        self.person_area = self.get_parameter("person_area").value
        self.emg_dist = self.get_parameter("emergency_dist").value

        self.model = YOLO("yolo11n.pt")
        self.bridge = CvBridge()
        self.create_subscription(Image, "/camera/image_raw", self.on_image, 10)
        self.create_subscription(LaserScan, "/scan", self.on_scan, qos_profile_sensor_data)
        self.pub = self.create_publisher(Twist, "/cmd_vel", 10)
        self.pub_dbg = self.create_publisher(Image, "/traffic/debug_image", 10)
        self.create_timer(0.1, self.control)   # 10Hz で制御

        self.state = "DRIVE"
        self.state_since = time.time()
        self.ignore_stop_until = 0.0
        self.seen = {"stop sign": 0.0, "person": 0.0}   # 面積比
        self.front_min = float("inf")

    # ---- センサ ----
    def on_image(self, msg):
        frame = self.bridge.imgmsg_to_cv2(msg, "bgr8")
        h, w = frame.shape[:2]
        res = self.model(frame, verbose=False, classes=[0, 11])[0]   # 0: person, 11: stop sign
        seen = {"stop sign": 0.0, "person": 0.0}
        for box in res.boxes:
            name = res.names[int(box.cls)]
            if float(box.conf) < 0.4:
                continue
            x1, y1, x2, y2 = box.xyxy[0].tolist()
            seen[name] = max(seen[name], (x2 - x1) * (y2 - y1) / (w * h))
```

```
        self.seen = seen
        self.pub_dbg.publish(self.bridge.cv2_to_imgmsg(res.plot(), "bgr8"))

    def on_scan(self, msg):
        r = np.array(msg.ranges)
        # 前方 ±20度（TurtleBot3 の LDS は index 0 が正面、反時計回り）
        front = np.concatenate([r[:20], r[-20:]])
        front = front[np.isfinite(front) & (front > 0.05)]
        self.front_min = float(front.min()) if len(front) else float("inf")

    # ---- 状態機械 ----
    def set_state(self, s):
        if s != self.state:
            self.get_logger().info(f"{self.state} -> {s}")
            self.state, self.state_since = s, time.time()

    def control(self):
        now = time.time()
        elapsed = now - self.state_since

        if self.front_min < self.emg_dist:
            self.set_state("EMERGENCY")
        elif self.state == "EMERGENCY":
            self.set_state("DRIVE")

        if self.state == "DRIVE":
            if self.seen["person"] > self.person_area:
                self.set_state("YIELD")
            elif self.seen["stop sign"] > self.stop_area and now > self.ignore_stop_until:
                self.set_state("STOP_SIGN")
        elif self.state == "STOP_SIGN" and elapsed > 3.0:
            self.ignore_stop_until = now + 5.0
            self.set_state("DRIVE")
        elif self.state == "YIELD" and self.seen["person"] < self.person_area * 0.5:
            self.set_state("DRIVE")

        cmd = Twist()
        if self.state == "DRIVE":
            cmd.linear.x = self.speed
        self.pub.publish(cmd)


def main():
    rclpy.init()
    node = TrafficRules()
    try:
        rclpy.spin(node)
    except KeyboardInterrupt:
        pass
```

```
    node.pub.publish(Twist())
    rclpy.shutdown()
```

ここから細かく解説します。

- `classes=[0, 11]`
- COCO のクラス番号で、人（0）と停止標識（11）だけを検出させます。関係ないクラスの誤検出を防ぎ、少し速くもなります。
- **制御を画像のコールバックから分けている理由**
- 画像は YOLO の推論の分だけ遅れて届きます。制御は 10Hz のタイマーで一定の周期で回し、**最新の認識結果を参照する**形にしています。こうすると LiDAR による緊急停止が画像の遅れに引きずられません。
- **YIELD の解除条件**
- 解除の閾値を開始の閾値の半分にしています（`person_area * 0.5`）。これは**ヒステリシス**と呼ばれる工夫で、閾値の境目で発進と停止を繰り返す「ガタガタ」を防ぎます。
- `ignore_stop_until`
- 停止して再発進した直後は、同じ標識がまだ見えています。5秒間無視しないと、永遠に止まり続けてしまいます。
- **LiDAR の正面**
- TurtleBot3 の LDS は、index 0 が正面です。他のロボットでは `angle_min` から正面の index を計算してください。

setup.py の entry_points には `'traffic_node = traffic_rules.traffic_node:main'` を登録します。

実行

```
# 端末1: 空のワールド
ros2 launch turtlebot3_gazebo empty_world.launch.py
```

Gazebo の Insert タブから、ロボットの前方の直線上に次のものを置きます。

- 2m先に Stop Sign（標識がロボットの方を向くように回転させる）
- 4m先の少し横に Person Standing

```
# 端末2
ros2 run traffic_rules traffic_node
# 端末3
ros2 run rqt_image_view rqt_image_view /traffic/debug_image
```

実行結果の見方

1. 標識の手前で止まり、ログに `DRIVE -> STOP_SIGN` が出て、3秒後に発進します。
2. 人が見えると `YIELD` で止まります。GUI で人を横にどかすと発進します。
3. 箱（Insert → Box）を正面に置くと、`EMERGENCY` で止まります。

ログの状態遷移とデバッグ画像を並べて画面録画すると、わかりやすい動画になります。

工夫ポイント（最終課題として発展させる案）

1. **白線に沿った走行** - TurtleBot3 AutoRace の白線追従（OpenCV）と組み合わせて、コースを走らせます。
2. **信号機の認識** - COCO の `traffic light` クラスで検出し、枠の中を HSV で見て赤か青かを判定します。

3. **距離の推定** - 停止標識の実際の大きさとカメラの焦点距離から距離を計算し、「標識の0.5m手前で止まる」ようにします。
4. **評価** - 標識の位置を変えて10回走らせ、停止位置のばらつきと、見逃した回数を記録します。

参考

- YOLO-ROS-2（Tinker-Twins）: https://github.com/Tinker-Twins/YOLO-ROS-2
- TurtleBot3 e-Manual: https://emanual.robotis.com/docs/en/platform/turtlebot3/simulation/
- TurtleBot3 AutoRace: https://emanual.robotis.com/docs/en/platform/turtlebot3/autonomous_driving/
- gazebo_models: https://github.com/osrf/gazebo_models
- COCO クラス一覧（Ultralytics）: https://docs.ultralytics.com/datasets/detect/coco/

目次へ

B01 ACT（ALOHA）を公式コードで再現する

環境

- OS: Ubuntu 20.04 / 22.04
- Python: 3.8.10（公式 README の指定）
- シミュレータ: MuJoCo 2.3.7 + dm_control 1.0.14
- GPU: VRAM 8GB以上
- 論文: Tony Z. Zhao, Vikash Kumar, Sergey Levine, Chelsea Finn, "Learning Fine-Grained Bimanual Manipulation with Low-Cost Hardware", RSS 2023
- コード: https://github.com/tonyzhaozh/act
- プロジェクトページ: https://tonyzhaozh.github.io/aloha/

概要

論文のポイント

ACT（Action Chunking with Transformers）は、人のデモから両腕の細かい作業を学ぶ模倣学習の手法です。

1. **Action Chunking（行動チャンキング）** - 1ステップずつではなく、**k ステップ分の行動をまとめて予測**します（論文では k=100）。 - 1ステップずつだと誤差が積み重なりますが（compounding error）、まとめて予測すると判断の回数が 1/k に減るので、誤差が溜まりにくくなります。
2. **Temporal Ensembling（時間方向のアンサンブル）** - 毎ステップ予測を行い、同じ時刻に対する過去の複数の予測を、指数関数的な重みで平均します。動きがなめらかになります。
3. **CVAE（条件付き変分オートエンコーダ）** - 人のデモには「同じ状況でも動き方が少しずつ違う」というばらつきがあります。これを潜在変数 z で吸収します。 - 推論時は z=0（平均）を使います。

この記事でやること

- 公式コードで、シミュレーションの2つのタスクを再現します。
- `sim_transfer_cube_scripted`：片手でキューブを持ち上げ、もう片方の手に渡す。
- `sim_insertion_scripted`：ペグを穴に挿入する。
- 論文の主張（チャンキングと Temporal Ensembling の効果）を、自分で**アブレーション実験**して確かめます。

準備

```
git clone https://github.com/tonyzhaozh/act.git
cd act
conda create -n aloha python=3.8.10 -y
conda activate aloha
pip install torchvision torch pyquaternion pyyaml rospkg pexpect \
  mujoco==2.3.7 dm_control==1.0.14 opencv-python matplotlib einops packaging h5py ipython
cd detr && pip install -e . && cd ..
```

- `detr/` には、ACT が内部で使う Transformer（DETR を改造したもの）が入っています。
- 描画で `GLFW` のエラーが出る場合は、`export MUJOCO_GL=egl` を設定します。

ソースコードの構成

- `record_sim_episodes.py`
- スクリプト化された（=プログラムで書かれた完璧な）方策でデモを生成し、HDF5 で保存します。
- `imitate_episodes.py`
- 学習と評価の本体です。`--eval` を付けると評価モードになります。
- `policy.py`
- `ACTPolicy`（CVAE + Transformer）と `CNNMLPPolicy`（比較用）が定義されています。
- `detr/models/detr_vae.py`
- ACT のネットワーク本体です。エンコーダが z を推定し、デコーダが k 個の行動を出力します。
- `sim_env.py` / `ee_sim_env.py`
- MuJoCo 環境（関節空間 / 手先空間）です。
- `constants.py`
- タスクごとの設定（データの場所、エピソード長、カメラ名）です。
- `visualize_episodes.py`
- 記録したエピソードを動画にします。

`constants.py` の `DATA_DIR` を自分のパスに書き換えておきます。

```
DATA_DIR = '/home/<user>/act_data'
```

実行

1. デモを生成する（50エピソード）

```
python3 record_sim_episodes.py \
  --task_name sim_transfer_cube_scripted \
  --dataset_dir /home/<user>/act_data/sim_transfer_cube_scripted \
  --num_episodes 50
```

`--onscreen_render` を付けると、生成中の様子が見られます。

2. デモを確認する

```
python3 visualize_episodes.py --dataset_dir /home/<user>/act_data/sim_transfer_cube_scripted --episode_idx 0
```

3. 学習

```
python3 imitate_episodes.py \
  --task_name sim_transfer_cube_scripted \
  --ckpt_dir /home/<user>/act_ckpt/transfer_cube_k100 \
  --policy_class ACT --kl_weight 10 --chunk_size 100 --hidden_dim 512 --batch_size 8 \
  --dim_feedforward 3200 --num_epochs 2000 --lr 1e-5 --seed 0
```

ここから細かく解説します。

- **`--chunk_size 100`**

- k の値です。アブレーション実験ではここを変えます。
- `--kl_weight 10`
- CVAE の KL 項の重みです（論文の β）。
- `--num_epochs 2000`
- README の推奨値です。RTX 3060 で数時間かかります。
- 時間がない場合は 500 程度でも、ある程度の成功率が出ます。最終課題では、エポック数もレポートに明記してください。

4. 評価

```
python3 imitate_episodes.py \
  --task_name sim_transfer_cube_scripted \
  --ckpt_dir /home/<user>/act_ckpt/transfer_cube_k100 \
  --policy_class ACT --kl_weight 10 --chunk_size 100 --hidden_dim 512 --batch_size 8 \
  --dim_feedforward 3200 --num_epochs 2000 --lr 1e-5 --seed 0 \
  --eval --temporal_agg
```

- `--temporal_agg`
- Temporal Ensembling を有効にします。
- **出力**
- `ckpt_dir` に、成功率（`result_policy_best.txt`）と評価動画（`video*.mp4`）が出ます。
- README によると、transfer cube は約90%、insertion は約50%前後が目安です。

アブレーション実験（最終課題の本体）

論文の Table / Figure にある「チャンクサイズと成功率の関係」を、小さな規模で再現します。

```
for k in 1 10 50 100; do
  python3 imitate_episodes.py --task_name sim_transfer_cube_scripted \
    --ckpt_dir /home/<user>/act_ckpt/k$k --policy_class ACT --kl_weight 10 --chunk_size $k \
    --hidden_dim 512 --batch_size 8 --dim_feedforward 3200 --num_epochs 500 --lr 1e-5 --seed 0
  python3 imitate_episodes.py --task_name sim_transfer_cube_scripted \
    --ckpt_dir /home/<user>/act_ckpt/k$k --policy_class ACT --kl_weight 10 --chunk_size $k \
    --hidden_dim 512 --batch_size 8 --dim_feedforward 3200 --num_epochs 500 --lr 1e-5 --seed 0 --eval
done
```

まとめる表の例（レポートに貼る用）：

- chunk_size = 1 / 10 / 50 / 100
- temporal_agg なし / あり
- 成功率（50エピソード評価）

比較用のベースライン

`--policy_class CNNMLP` を指定すると、チャンキングのない単純な行動クローニングと比べられます。

結果をグラフにする

```
import matplotlib.pyplot as plt
ks = [1, 10, 50, 100]
success = [0.0, 0.0, 0.0, 0.0]   # result_policy_best.txt の値を入れる
plt.plot(ks, success, marker="o")
plt.xscale("log"); plt.xlabel("chunk size k"); plt.ylabel("success rate")
plt.title("ACT: effect of action chunking (transfer cube, 500 epochs)")
plt.savefig("chunk_ablation.png", dpi=150)
```

工夫ポイント（最終課題として発展させる案）

1. **デモ数の効果** - デモを 10 / 25 / 50 に変えて学習します（`constants.py` の `num_episodes`）。
2. **ノイズへの頑健性** - 評価時に、キューブの初期位置の範囲を広げます（`sim_env.py` の `sample_box_pose`）。学習時の分布の外での性能を見ます。
3. **自分のタスクを作る** - `scripted_policy.py` を真似して、新しいスクリプト方策（例: キューブを2段に積む）を書き、デモ生成から学習までを一通り行います。独自性が大きく上がります。
4. **LeRobot 版との比較** - A04 の LeRobot の ACT と結果を比べ、実装の違いを考察します。

参考

- 論文: https://arxiv.org/abs/2304.13705
- コード: https://github.com/tonyzhaozh/act
- プロジェクトページ: https://tonyzhaozh.github.io/aloha/
- LeRobot の ACT: https://github.com/huggingface/lerobot

目次へ

B02 Diffusion PolicyをPushTで再現し、ACTと比較する

環境

- OS: Ubuntu 20.04 / 22.04
- Python: conda 環境（`conda_environment.yaml` で自動構築、`robodiff`）
- シミュレータ: PushT（pymunk による2D物理）
- GPU: VRAM 12GB以上を推奨（画像入力で学習する場合）
- 論文: Cheng Chi et al., "Diffusion Policy: Visuomotor Policy Learning via Action Diffusion", RSS 2023（IJRR 2024 に拡張版）
- コード: https://github.com/real-stanford/diffusion_policy
- プロジェクトページ: https://diffusion-policy.cs.columbia.edu/

概要

論文のポイント

Diffusion Policy は、画像生成で使われる**拡散モデル（DDPM）**を、行動の生成に使います。

1. ランダムなノイズから始めて、観測（画像と状態）を条件に、**ノイズを少しずつ取り除いて**行動列を生成します。
2. **マルチモーダル性への強さ** - 「T字ブロックを左から回り込んでも右から回り込んでもよい」場合、平均を取る方式（通常の回帰）では、真ん中に突っ込む失敗をします。 - 拡散モデルは、どちらか一方のモードをきちんと選べます。
3. ACT と同じく**行動列（action horizon）をまとめて予測**し、receding horizon（予測の一部だけを実行して再予測する方式）で実行します。

この記事でやること

1. 公式コードで、PushT の学習済みモデルを評価する（すぐに動く）。
2. 公式コードで、画像入力の Diffusion Policy を学習する。
3. **LeRobot で ACT と Diffusion Policy を同じ条件で学習して比較する**（最終課題の本体）。
4. マウスで自分のデモを追加する（公式のデモツール）。

準備

```
sudo apt install -y libosmesa6-dev libgl1-mesa-glx libglfw3 patchelf
git clone https://github.com/real-stanford/diffusion_policy.git
cd diffusion_policy
# mamba がなければ conda でも可（時間がかかる）
mamba env create -f conda_environment.yaml
conda activate robodiff
```

データのダウンロード

```
mkdir -p data && cd data
wget https://diffusion-policy.cs.columbia.edu/data/training/pusht.zip
```

```
unzip pusht.zip && rm -f pusht.zip && cd ..
```

実行1: 公式の設定で学習する

```
# 公式の画像版 CNN 設定をダウンロード
wget -O image_pusht_diffusion_policy_cnn.yaml \
  https://diffusion-policy.cs.columbia.edu/data/experiments/image/pusht/diffusion_policy_cnn/config.yaml

python train.py --config-dir=. --config-name=image_pusht_diffusion_policy_cnn.yaml \
  training.seed=42 training.device=cuda:0 \
  hydra.run.dir='data/outputs/${now:%Y.%m.%d}/${now:%H.%M.%S}_${name}_${task_name}'
```

ここから細かく解説します。

- **Hydra**
- 設定は Hydra の YAML で管理されています。`key=value` の形でコマンドラインから上書きできます（例: `training.num_epochs=500`）。
- **主な設定項目**（YAML の中）
- `horizon: 16`: 予測する行動列の長さ
- `n_obs_steps: 2`: 入力に使う過去の観測の数
- `n_action_steps: 8`: 予測のうち実際に実行する数（receding horizon）
- `policy.noise_scheduler.num_train_timesteps: 100`: 拡散のステップ数
- **wandb**
- 公式は wandb にログを送る設定です。使わない場合は `logging.mode=offline` を付けます。
- **学習時間**
- 公式の設定（3000エポック）は非常に長いです。課題では `training.num_epochs=300` 程度に減らし、その旨をレポートに書きます。

実行2: 評価

```
python eval.py --checkpoint data/outputs/<日付>/<時刻>_.../checkpoints/latest.ckpt \
  --output_dir data/pusht_eval_output --device cuda:0
```

- `data/pusht_eval_output/eval_log.json` に、`test/mean_score`（ブロックと目標の重なり率の平均、0〜1）が出ます。
- `media/` に評価動画が出ます。
- 学習済みのチェックポイントは、プロジェクトページの「Data」から入手できます。まずはそれで評価だけを試してみるとよいでしょう。

実行3: マウスで自分のデモを集める

```
python demo_pusht.py -o data/my_pusht_demo.zarr
```

ウィンドウの中でマウスを動かすとエージェントが動き、T字ブロックを押せます。自分で集めたデモで学習させると、「自分で教えたロボット」として独自性が出ます。

実行4: ACT との比較（LeRobot で同条件にそろえる）

公式の ACT（B01）は ALOHA 用、公式の Diffusion Policy は PushT 用で、コードの土台が違います。そのため、**LeRobot で同じデータセット・同じステップ数にそろえて**比べるのがおすすめです。環境構築は A04 を参照してください。

```
# Diffusion Policy
lerobot-train --policy.type=diffusion --env.type=pusht --dataset.repo_id=lerobot/pusht \
  --output_dir=outputs/cmp/diffusion --steps=50000 --eval_freq=10000 \
  --policy.device=cuda --policy.push_to_hub=false

# ACT
lerobot-train --policy.type=act --env.type=pusht --dataset.repo_id=lerobot/pusht \
  --output_dir=outputs/cmp/act --steps=50000 --eval_freq=10000 \
  --policy.device=cuda --policy.push_to_hub=false

# 評価（各50エピソード）
for m in diffusion act; do
  lerobot-eval --policy.path=outputs/cmp/$m/checkpoints/last/pretrained_model \
    --env.type=pusht --eval.n_episodes=50 --eval.batch_size=10 --policy.device=cuda
done
```

比較する観点

1. **成功率と平均報酬**: `eval_info.json` の値を比べます。
2. **マルチモーダル性** - 同じ初期配置で10回実行し、エージェントの軌跡を重ねて描きます。 - Diffusion Policy は「左回り」と「右回り」に分かれ、平均を取る方式は迷いやすい、という違いが見えれば論文の主張を再現できたことになります。
3. **推論速度** - 拡散モデルは推論のステップ数が多いので遅くなります。1行動あたりの推論時間（ms）を計測します。 - DDIM でステップ数を減らす（`num_inference_steps` を 100 → 10）とどうなるかも見ます。

軌跡を重ねて描くスクリプト（概略）

```
import gymnasium as gym, gym_pusht, numpy as np, torch, matplotlib.pyplot as plt
from lerobot.policies.diffusion.modeling_diffusion import DiffusionPolicy  # 版により lerobot.common.policies...

policy = DiffusionPolicy.from_pretrained("outputs/cmp/diffusion/checkpoints/last/pretrained_model").to("cuda")
env = gym.make("gym_pusht/PushT-v0", obs_type="pixels_agent_pos")
for trial in range(10):
    obs, _ = env.reset(seed=0)          # 同じ初期配置
    policy.reset()
    xs = []
    for t in range(300):
        batch = {
            "observation.state": torch.tensor(obs["agent_pos"], dtype=torch.float32)[None].cuda(),
            "observation.image": torch.tensor(obs["pixels"]).permute(2, 0, 1)[None].float().cuda() / 255,
        }
        with torch.no_grad():
            a = policy.select_action(batch)[0].cpu().numpy()
        obs, r, term, trunc, _ = env.step(a)
```

```
        xs.append(obs["agent_pos"])
        if term or trunc:
            break
    xs = np.array(xs)
    plt.plot(xs[:, 0], xs[:, 1], alpha=0.6)
plt.gca().invert_yaxis(); plt.title("Diffusion Policy trajectories (seed=0, 10 runs)")
plt.savefig("traj_diffusion.png", dpi=150)
```

- 入力キーの名前は、データセットの features（`observation.image`、`observation.state`）に合わせます。
- 同じことを ACT でも行い、2枚の図を並べます。

工夫ポイント（最終課題として発展させる案）

1. **推論ステップ数と性能のトレードオフ**: DDPM 100ステップ / DDIM 10ステップ / DDIM 5ステップで比べます。
2. **デモの質**: 自分でマウスで集めた雑なデモと、公式のデモとで学習結果を比べます。
3. **horizon / n_action_steps のアブレーション**: 予測の長さと再予測の頻度が性能に与える影響を調べます。

参考

- 論文: https://arxiv.org/abs/2303.04137
- コード: https://github.com/real-stanford/diffusion_policy
- プロジェクトページ: https://diffusion-policy.cs.columbia.edu/
- LeRobot: https://github.com/huggingface/lerobot
- gym-pusht: https://github.com/huggingface/gym-pusht

目次へ

B03 legged_gymの地形カリキュラムをGenesisに移植する

環境

- OS: Ubuntu 22.04
- Python: 3.10
- シミュレータ: Genesis（A03 の環境）
- ロボット: Unitree Go2
- 強化学習: PPO（rsl-rl-lib）
- 論文: Nikita Rudin, David Hoeller, Philipp Reist, Marco Hutter, "Learning to Walk in Minutes Using Massively Parallel Deep Reinforcement Learning", CoRL 2021
- コード: https://github.com/leggedrobotics/legged_gym
- 関連論文: Tobin et al., "Domain Randomization for Transferring Deep Neural Networks from Simulation to the Real World", IROS 2017

概要

論文のポイント

legged_gym は、GPU 上で数千台のロボットを同時にシミュレーションし、**数分〜20分で歩行を学習させる**ことを示した研究です。中でも重要なのが**地形カリキュラム**です。

- 地形を「難易度（行）× 種類（列）」の格子状に並べます（平地、坂、階段、飛び石、ランダムな凸凹）。
- 各ロボットは、最初は簡単な行（level 0）から始めます。
- エピソードの終わりに、**十分遠くまで歩けたら難易度を1段上げ、ほとんど歩けなかったら1段下げます。**
- 一番難しい段をクリアしたロボットは、ランダムな難易度に戻されます（忘れないようにするため）。

これに、摩擦のランダム化や外乱（Domain Randomization）を組み合わせて、実機でも動く方策を作っています。

この記事でやること

legged_gym 本体は **Isaac Gym Preview 4（開発終了・Python 3.8）** が必要で、今から環境を作るのは大変です。そこで、**legged_gym のコードを読んでカリキュラムの仕組みを理解し、それを Genesis の Go2 サンプル（A03）に移植**します。

- 移植元: `legged_gym/utils/terrain.py`（地形の生成）と `legged_gym/envs/base/legged_robot.py` の `_update_terrain_curriculum()`
- 移植先: `Genesis/examples/locomotion/go2_env.py`

準備

```
conda activate genesis      # A03 の環境
git clone https://github.com/leggedrobotics/legged_gym.git   # 読むだけ
```

移植元のコードを読む

`legged_gym/envs/base/legged_robot.py` の中心部分（要約）は次のとおりです。

```
def _update_terrain_curriculum(self, env_ids):
    distance = torch.norm(self.root_states[env_ids, :2] - self.env_origins[env_ids, :2], dim=1)
    # 地形の半分以上を歩けたら難しくする
    move_up = distance > self.terrain.env_length / 2
    # 指令速度から期待される距離の半分も歩けなかったら易しくする
    move_down = (distance < torch.norm(self.commands[env_ids, :2], dim=1) * self.max_episode_length_s * 0.5) * ~move_up
    self.terrain_levels[env_ids] += 1 * move_up - 1 * move_down
    # 最難関を超えたらランダムな難易度へ
    self.terrain_levels[env_ids] = torch.where(
        self.terrain_levels[env_ids] >= self.max_terrain_level,
        torch.randint_like(self.terrain_levels[env_ids], self.max_terrain_level),
        torch.clip(self.terrain_levels[env_ids], 0))
    self.env_origins[env_ids] = self.terrain_origins[self.terrain_levels[env_ids], self.terrain_types[env_ids]]
```

- `env_origins`：そのロボットが出発する地点（＝担当する地形の区画の中心）です。
- `terrain_origins[level, type]`：地形の格子の各区画の中心座標です。

ソースコード（Genesis への移植）

1. 地形を格子状に作る（go2_env.py の `__init__` 内）

元の `self.scene.add_entity(gs.morphs.Plane())`（または URDF の平面）を、次のように地形に置き換えます。

```
        # ---- 地形カリキュラム用の地形（行: 難易度 / 列: 種類） ----
        self.n_levels, self.n_types = 5, 4
        self.sub_size = 6.0   # 1区画 6m x 6m
        types_per_col = ["flat_terrain", "pyramid_sloped_terrain",
                         "random_uniform_terrain", "pyramid_stairs_terrain"]
        subterrain_types = [[types_per_col[c] if r > 0 else "flat_terrain"
                             for c in range(self.n_types)] for r in range(self.n_levels)]
        self.terrain = self.scene.add_entity(
            gs.morphs.Terrain(
                n_subterrains=(self.n_levels, self.n_types),
                subterrain_size=(self.sub_size, self.sub_size),
                horizontal_scale=0.1,
                vertical_scale=0.005,
                subterrain_types=subterrain_types,
            )
        )
```

注意:

- Genesis の `Terrain` には難易度のパラメータが直接ないことがあります。その場合は次のどちらかにします。
- 「行が下がるほど険しい種類」になるように `subterrain_types` を並べる（上の例では level 0 を平地にしています）。
- `height_field` に自分で生成した高さマップを渡す。
- 引数の名前と、使える `subterrain_types` は Genesis の版によって違います。`help(gs.morphs.Terrain)` で確認してください。

2. 各ロボットの難易度と出発地点（`__init__` の最後）

```
    # 区画の中心座標 [level, type, xyz]
    i = torch.arange(self.n_levels, device=self.device)
    j = torch.arange(self.n_types, device=self.device)
    ii, jj = torch.meshgrid(i, j, indexing="ij")
    self.terrain_origins = torch.stack([
        (ii.float() + 0.5) * self.sub_size,
        (jj.float() + 0.5) * self.sub_size,
        torch.zeros_like(ii, dtype=torch.float)], dim=-1)

    self.terrain_levels = torch.zeros(self.num_envs, dtype=torch.long, device=self.device)
    self.terrain_types = torch.arange(self.num_envs, device=self.device) % self.n_types
    self.env_origins = self.terrain_origins[self.terrain_levels, self.terrain_types]
```

3. リセット時にカリキュラムを更新して、出発地点を変える（`reset_idx` 内）

```
def _update_terrain_curriculum(self, envs_idx):
    if len(envs_idx) == 0:
        return
    dist = torch.norm(self.base_pos[envs_idx, :2] - self.env_origins[envs_idx, :2], dim=1)
    cmd_dist = torch.norm(self.commands[envs_idx, :2], dim=1) * self.env_cfg["episode_length_s"]
    move_up = dist > self.sub_size / 2
    move_down = (dist < cmd_dist * 0.5) & ~move_up
    lv = self.terrain_levels[envs_idx] + move_up.long() - move_down.long()
    lv = torch.where(lv >= self.n_levels,
                     torch.randint_like(lv, self.n_levels),
                     torch.clamp(lv, min=0))
    self.terrain_levels[envs_idx] = lv
    self.env_origins[envs_idx] = self.terrain_origins[lv, self.terrain_types[envs_idx]]

def reset_idx(self, envs_idx):
    self._update_terrain_curriculum(envs_idx)       # ← 追加
    # ...（元の reset_idx の処理）...
    # base_pos の初期値を env_origins 基準にする
    self.base_pos[envs_idx] = self.base_init_pos + self.env_origins[envs_idx]
    self.base_pos[envs_idx, 2] += 0.1               # 地形の凹凸に埋まらないよう少し上から
    self.robot.set_pos(self.base_pos[envs_idx], zero_velocity=False, envs_idx=envs_idx)
    # ...
    # ログ用
    self.extras["episode"]["terrain_level"] = self.terrain_levels.float().mean()
```

ここから細かく解説します。

- `self.extras["episode"]`
- rsl-rl はここに入れた値を TensorBoard に書き出します。**平均の地形レベルが学習とともに上がっていくグラフ**は、カリキュラムが効いている証拠になるので、レポートに必ず入れましょう。
- **地形の上での高さ**
- 平地と違って、凸凹の上で `base_pos[2]` がずれるため、少し上から落とします。

- 終了判定に「胴体の高さが低すぎる」がある場合、地形の高さを基準にしないと誤判定になります。必要に応じて閾値を緩めてください。
- `episode_length_s`
- A03 の `env_cfg` にある値です。名前が違う場合は合わせてください。

4. Domain Randomization（摩擦のランダム化）

```
        # __init__ の scene.build() 後に追加（API名は版によって違うので docs で確認）
        friction = gs_rand_float(0.4, 1.25, (self.num_envs, 1), self.device)
        self.robot.set_friction_ratio(friction.repeat(1, self.robot.n_links),
                                      torch.arange(self.robot.n_links, device=self.device))
```

5. 観測に足りないもの

legged_gym の荒れた地形用の方策は、**周囲の高さマップ（height scan）**を観測に入れています。今回は省略して、**盲目（blind）の歩行**として学習させます。高さマップの追加は発展課題です。盲目でも、坂や低い段差ならある程度歩けます。

実行

```
cd Genesis/examples/locomotion
python go2_train.py -e go2-terrain -B 4096 --max_iterations 1500
tensorboard --logdir logs   # Episode/terrain_level を確認
python go2_eval.py -e go2-terrain --ckpt 1499
```

- 地形があると学習が難しくなるので、反復回数を 1000〜1500 に増やします。
- RTX 3060 で1〜2時間が目安です。

比較実験（最終課題の本体）

1. **カリキュラムあり / なし** - なしの場合は、最初からランダムな難易度に置きます。 - 両者で、学習曲線と最終的な平均地形レベルを比べます。
2. **平地だけで学習した方策（A03）を、地形の上で評価** - 転倒率を比べ、汎化性能の差を示します。
3. **摩擦のランダム化あり / なし** - 評価時の摩擦を 0.3（滑りやすい）にして比べます。

参考

- 論文: https://arxiv.org/abs/2109.11978
- legged_gym: https://github.com/leggedrobotics/legged_gym
- rsl_rl: https://github.com/leggedrobotics/rsl_rl
- Genesis Terrain（API）: https://genesis-world.readthedocs.io/
- Tobin et al., Domain Randomization: https://arxiv.org/abs/1703.06907

目次へ

B04 Eurekaの仕組みでLLMに報酬関数を書かせる

環境

- OS: Ubuntu 22.04
- Python: 3.10
- シミュレータ: Genesis（A03 の Go2 環境）
- 強化学習: PPO（rsl-rl-lib）
- LLM: Ollama（`qwen2.5-coder:7b` など、コード生成が得意なモデル）
- API キーがあれば、GPT-4 クラスのモデルの方が成功率は上がります。
- 論文: Yecheng Jason Ma et al., "Eureka: Human-Level Reward Design via Coding Large Language Models", ICLR 2024
- コード: https://github.com/eureka-research/Eureka
- プロジェクトページ: https://eureka-research.github.io/

概要

論文のポイント

強化学習で一番難しいのは**報酬関数の設計**です。Eureka は LLM にそれをやらせます。

1. **環境のソースコードをそのまま LLM に渡す** - 観測にどんな変数があるかを、LLM が自分で読み取ります。
2. **K 個の報酬関数を生成**し、それぞれで短時間学習させます。
3. 一番良かったものについて、**報酬の各項の推移（統計）を文章にして LLM に返します**（Reward Reflection）。 - 例:「tracking 項は学習中ずっと 0.01 で変化なし」
4. LLM がそれを読んで報酬関数を改良します。これを N 回繰り返します。

論文では、29個のタスクのうち83%で、人間の専門家が書いた報酬を上回りました。

この記事でやること

Eureka 本体は **Isaac Gym** が前提です（`python eureka.py env=shadow_hand sample=4 iteration=2 model=gpt-4-0314` のように実行します）。今から Isaac Gym の環境を作るのは大変なので、**同じ仕組みを Genesis の Go2 で作る「ミニEureka」**を作ります。

- タスク: Go2 に**後ろ歩き**（または「その場で回転」「ジャンプ」）をさせる。人が報酬を書いていない新しい動きです。
- 1回の候補の学習を短く（100反復・数分）して、K=3、N=3 で回します。

処理の流れは次のとおりです。

1. LLM に、タスクの説明と `go2_env.py` の抜粋を渡して、`compute_reward(env)` のコードを K 個書かせる。
2. 候補ごとに次を行う。
3. `custom_reward_k.py` として保存する。
4. `go2_train.py --custom_reward` で短時間学習する。
5. `fitness.py` で適応度（タスク達成度）を測る。
6. 報酬の各項の統計を集める。
7. 一番良い候補と、その統計の文章を LLM に返す（Reward Reflection）。

8. これを N 回繰り返す。

準備

```
conda activate genesis   # A03 の環境
pip install ollama
ollama pull qwen2.5-coder:7b
```

ソースコード

1. go2_env.py にカスタム報酬の差し込み口を作る

```
# go2_env.py の __init__ の最後（reward_functions を作った後）に追加
        import importlib.util
        path = env_cfg.get("custom_reward_path")
        if path:
            spec = importlib.util.spec_from_file_location("custom_reward", path)
            mod = importlib.util.module_from_spec(spec)
            spec.loader.exec_module(mod)
            self._custom_fn = mod.compute_reward
            self.custom_stats = {}
        else:
            self._custom_fn = None
```

```
# go2_env.py の step() 内、既存の報酬計算ループの直後に追加
        if self._custom_fn is not None:
            total, comps = self._custom_fn(self)
            self.rew_buf[:] = total * self.dt          # Eureka 方式では既存の報酬を置き換える
            for k, v in comps.items():                 # 各項の平均を記録（Reflection 用）
                self.custom_stats.setdefault(k, []).append(float(v.mean()))
```

- `self.rew_buf[:] = ...`
- 既存の報酬を**置き換えて**います。足し合わせる場合は `+=` にします。
- 「人の報酬 + LLM の報酬」と「LLM の報酬だけ」を比べるのも面白い実験です。
- `custom_stats`
- 報酬の各項の平均をステップごとに記録し、学習の最後に LLM に返します。

2. go2_train.py に引数を追加する

```
    parser.add_argument("--custom_reward", type=str, default=None)
    parser.add_argument("--command", type=float, nargs=3, default=None)   # 例: -0.5 0 0 で後ろ歩き
    # ...
    env_cfg["custom_reward_path"] = args.custom_reward
    if args.command is not None:
        command_cfg["lin_vel_x_range"] = [args.command[0]] * 2
        command_cfg["lin_vel_y_range"] = [args.command[1]] * 2
        command_cfg["ang_vel_range"] = [args.command[2]] * 2
```

```
    # 学習後に統計を保存
    # runner.learn(...) の後に:
    import json
    stats = {k: [float(sum(v[i:i+200]) / len(v[i:i+200])) for i in range(0, len(v), max(1, len(v)//10))]
             for k, v in env.custom_stats.items()} if args.custom_reward else {}
    json.dump(stats, open(f"{log_dir}/reward_stats.json", "w"))
```

3. 適応度（タスク達成度）を測る: fitness.py

LLM が書いた報酬の値そのものではなく、**人が決めた客観的な指標**で候補を比べます。これが Eureka の重要なポイントです。

```
# fitness.py: 学習済み方策を走らせ、平均の前後方向の速度を返す（後ろ歩きなら負が良い）
import argparse, pickle, torch, genesis as gs
from rsl_rl.runners import OnPolicyRunner
from go2_env import Go2Env

def main():
    p = argparse.ArgumentParser()
    p.add_argument("-e", "--exp_name", required=True)
    p.add_argument("--ckpt", type=int, required=True)
    a = p.parse_args()
    gs.init(logging_level="warning")
    log_dir = f"logs/{a.exp_name}"
    env_cfg, obs_cfg, reward_cfg, command_cfg, train_cfg = pickle.load(open(f"{log_dir}/cfgs.pkl", "rb"))
    env = Go2Env(num_envs=64, env_cfg=env_cfg, obs_cfg=obs_cfg,
                 reward_cfg=reward_cfg, command_cfg=command_cfg, show_viewer=False)
    runner = OnPolicyRunner(env, train_cfg, log_dir, device="cuda:0")
    runner.load(f"{log_dir}/model_{a.ckpt}.pt")
    policy = runner.get_inference_policy(device="cuda:0")
    obs, _ = env.reset()
    vx, falls = [], 0
    with torch.no_grad():
        for _ in range(500):
            obs, _, dones, _ = env.step(policy(obs))
            vx.append(env.base_lin_vel[:, 0].mean().item())
            falls += int(dones.sum().item())
    print(f"FITNESS vx_mean={sum(vx)/len(vx):.4f} falls={falls}")

if __name__ == "__main__":
    main()
```

- `cfgs.pkl` に保存されている要素の数や、`OnPolicyRunner` の引数は、`go2_eval.py` の書き方に合わせてください。
- 評価時は `custom_reward_path` を空にしても構いません（報酬は使わないため）。

4. Eureka のループ: mini_eureka.py

```
import json, re, subprocess
from pathlib import Path
```

```
import ollama

MODEL = "qwen2.5-coder:7b"
K, N_ITER, TRAIN_ITERS = 3, 3, 100
TASK = "Go2 四脚ロボットを、転ばずに後ろ向き（-x方向）に 0.5 m/s で歩かせる。"
ENV_SRC = Path("go2_env.py").read_text()[:6000]   # 長すぎる場合は観測まわりの抜粋を渡す

SYSTEM = """あなたは強化学習の報酬設計の専門家です。
次の形式の Python 関数だけを ```python ``` で囲んで出力してください。
def compute_reward(env):
    # env の属性（例: env.base_lin_vel, env.base_ang_vel, env.projected_gravity,
    #   env.dof_pos, env.default_dof_pos, env.actions, env.last_actions, env.base_pos）を使う
    # 戻り値: (total: Tensor[num_envs], components: dict[str, Tensor[num_envs]])
torch は import 済みとして使ってよいが、関数内で import torch と書くこと。"""


def ask(messages):
    r = ollama.chat(model=MODEL, messages=messages, options={"temperature": 0.8})
    text = r["message"]["content"]
    m = re.search(r"```python\n(.*?)```", text, re.S)
    return m.group(1) if m else text


def run(cmd):
    out = subprocess.run(cmd, shell=True, capture_output=True, text=True)
    return out.stdout + out.stderr


def evaluate(k_dir, code):
    path = Path(k_dir) / "reward.py"
    path.parent.mkdir(parents=True, exist_ok=True)
    path.write_text(code)
    exp = f"eureka_{Path(k_dir).name}"
    log = run(f"python go2_train.py -e {exp} -B 2048 --max_iterations {TRAIN_ITERS} "
              f"--custom_reward {path} --command -0.5 0 0")
    if "Traceback" in log:
        return None, "実行時エラー:\n" + log[-1500:]
    fit = run(f"python fitness.py -e {exp} --ckpt {TRAIN_ITERS - 1}")
    m = re.search(r"vx_mean=(-?[\d.]+) falls=(\d+)", fit)
    stats = json.load(open(f"logs/{exp}/reward_stats.json"))
    score = -float(m.group(1)) - 0.001 * int(m.group(2)) if m else -1e9   # 後ろ向きに速いほど良い
    summary = "\n".join(f"{k}: " + ", ".join(f"{x:.3f}" for x in v) for k, v in stats.items())
    feedback = (f"適応度: vx_mean={m.group(1) if m else 'NA'} (目標 -0.5), 転倒回数={m.group(2) if m else 'NA'}\n"
                f"学習中の各報酬項の平均の推移:\n{summary}")
    return score, feedback


messages = [{"role": "system", "content": SYSTEM},
            {"role": "user", "content": f"タスク: {TASK}\n環境のソースコード:\n{ENV_SRC}"}]
```

```
best = (-1e9, None, None)
for it in range(N_ITER):
    results = []
    for k in range(K):
        code = ask(messages)
        score, fb = evaluate(f"eureka_runs/it{it}_k{k}", code)
        print(f"[iter {it} cand {k}] score={score}")
        results.append((score if score is not None else -1e9, code, fb))
    results.sort(key=lambda x: x[0], reverse=True)
    top = results[0]
    if top[0] > best[0]:
        best = top
    # Reward Reflection
    messages += [{"role": "assistant", "content": f"```python\n{top[1]}```"},
                 {"role": "user", "content": f"この報酬関数で学習した結果です。\n{top[2]}\n"
                  "結果を分析し、改良した compute_reward を出力してください。"
                  "変化していない項はスケールを変えるか作り直し、タスクに効いていない項は削除してください。"}]

Path("best_reward.py").write_text(best[1])
print("BEST SCORE", best[0])
print(best[2])
```

ここから細かく解説します。

- **適応度と報酬を分けている**
- LLM の報酬が「楽をして高得点を取る」抜け道を作っても、外部の適応度（実際の後ろ向きの速度）で評価するので騙されません。論文でいう fitness function です。
- **Reward Reflection**
- 「どの項がどう推移したか」を数値の列で返します。LLM は「`backward_vel` 項が増えていないのでスケールが小さすぎる」のように推論して改良します。
- **エラー処理**
- 生成されたコードが実行時エラーになったら、そのトレースバックをフィードバックとして返します。論文でもこの方式です。
- **計算コスト**
- K × N 回の学習が走ります。100反復×9回で、RTX 3060 なら1〜2時間です。
- `-B 2048` でも重い場合は、さらに減らしてください。

実行

```
cd Genesis/examples/locomotion
cp /path/to/fitness.py /path/to/mini_eureka.py .
python mini_eureka.py
# 最良の報酬で本学習
python go2_train.py -e go2-backward -B 4096 --max_iterations 500 --custom_reward best_reward.py --command -0.5 0 0
python go2_eval.py -e go2-backward --ckpt 499
```

比較実験（最終課題の本体）

1. **人が書いた報酬との比較** - 公式の `tracking_lin_vel` などの報酬で、指令を -0.5 にして学習したものと比べます。
2. **反復ごとの改善** - iter 0 / 1 / 2 の最良候補の適応度をグラフにします。Reflection の効果がわかります。
3. **LLM が書いた報酬関数そのもの** - コードをレポートに掲載し、人と違う発想があれば考察します。
4. **LLM の比較** - 7B のローカルLLM と大きなモデルとで、生成したコードの成功率（エラーにならない割合）を比べます。

参考

- 論文: https://arxiv.org/abs/2310.12931
- コード: https://github.com/eureka-research/Eureka
- プロジェクトページ: https://eureka-research.github.io/
- Genesis Locomotion: https://genesis-world.readthedocs.io/en/latest/user_guide/policy_training/examples/locomotion.html
- Ollama Python: https://github.com/ollama/ollama-python

目次へ

B05 Code as Policies: LLMが書いたコードで積み木を積む

環境

- OS: Ubuntu 22.04（Colab 可）
- Python: 3.10
- シミュレータ: Genesis + Franka（A07 の `franka_api.py` を使います）
- LLM: Ollama（`qwen2.5-coder:7b` など）。API キーがあれば GPT 系や Claude 系でも可。
- 論文: Jacky Liang et al., "Code as Policies: Language Model Programs for Embodied Control", ICRA 2023
- コード: https://github.com/google-research/google-research/tree/master/code_as_policies
- プロジェクトページ: https://code-as-policies.github.io/

概要

論文のポイント

Code as Policies（CaP）は、LLM に**ロボットの API を呼ぶ Python コードを書かせ、それを実行する**手法です。

- 「方策（policy）＝コード」と考えます。`for` 文や `if` 文、算術を使えるので、「3つを等間隔に並べて」「一番大きいものから」のような**空間的・論理的な指示**に強いです。
- **Few-shot プロンプト**として、「指示 → コード」の例をいくつか見せます。
- **階層的なコード生成**: 未定義の関数（例: `get_pyramid_positions()`）が出てきたら、その関数も LLM に書かせます。

この記事でやること

1. 公式の Colab デモ（PyBullet + UR5）で論文の挙動を確認する（任意）。
2. Genesis + Franka の上に**自分で CaP を実装**する。 - ロボットAPI: `get_obj_names()`、`get_obj_pos(name)`、`pick_place(obj, target_pos)` - 指示の例: 「赤・緑・青の順に積み上げて」「全部を一列に並べて」
3. **実行時エラーが出たら、トレースバックを LLM に返して直させる**（自己修正ループ）。

処理の流れは次のとおりです。

1. ユーザーが自然言語で指示する（例: 「青の上に赤を積んで」）。
2. Few-shot プロンプトと API の説明を付けて、LLM に送る。
3. LLM が Python コードを生成する（例: `pick_place('red_cube', above('blue_cube'))`）。
4. 制限付きの名前空間で `exec` し、FrankaAPI 経由で Genesis の Franka を動かす。
5. エラーが出たらトレースバックを LLM に返し、修正版のコードで再実行する。

準備

```
conda activate genesis   # A07 の環境（franka_api.py がある場所）
pip install ollama
ollama pull qwen2.5-coder:7b
```

（任意）公式デモ

https://github.com/google-research/google-research/tree/master/code_as_policies から `Interactive_Demo.ipynb` を Colab で開きます。

- OpenAI の API キーが必要です。
- 当時のモデル（Codex）は提供が終了しているので、ノートブックのモデル名を現行のものに書き換えてください。

ソースコード

cap_genesis.py

```
import re
import traceback
import numpy as np
import ollama
from franka_api import FrankaAPI, make_scene, to_np

MODEL = "qwen2.5-coder:7b"
CUBE = 0.04

OBJECTS = {
    "red_cube":   ((0.55,  0.15, 0.02), (1.0, 0.1, 0.1)),
    "green_cube": ((0.60, -0.05, 0.02), (0.1, 0.8, 0.1)),
    "blue_cube":  ((0.45, -0.20, 0.02), (0.1, 0.2, 1.0)),
}

scene, franka, ents, cam = make_scene(OBJECTS)
api = FrankaAPI(scene, franka, cam)


# ---- LLM に公開するロボットAPI ----
def get_obj_names():
    """机の上の物体名のリストを返す"""
    return list(ents.keys())


def get_obj_pos(name):
    """物体の中心座標 np.array([x, y, z]) [m] を返す"""
    return to_np(ents[name].get_pos()).copy()


def pick_place(obj_name, target_pos):
    """obj_name を掴んで、target_pos（物体中心を置きたい座標）に置く"""
    api.pick(get_obj_pos(obj_name))
    api.place(np.asarray(target_pos, dtype=float))


def above(name):
    """name の物体のちょうど上に積むための座標を返す"""
    p = get_obj_pos(name)
    return p + np.array([0, 0, CUBE])
```

```
TABLE = {"x_min": 0.35, "x_max": 0.70, "y_min": -0.30, "y_max": 0.30, "z": 0.02}

# ---- プロンプト ----
PROMPT_HEADER = f'''
# 次の Python API でロボットアームを制御するコードを書く。
# get_obj_names() -> list[str]
# get_obj_pos(name) -> np.ndarray [x, y, z]（物体中心, m）
# pick_place(obj_name, target_pos)   # target_pos は置いたときの物体中心
# above(name) -> np.ndarray          # name の上に積む位置
# TABLE = {TABLE}   # 置いてよい範囲。立方体の一辺は {CUBE} m
# np（numpy）は使える。コードだけを ```python``` で出力する。

# 指示: 赤い立方体を青い立方体の上に置いて。
```python
pick_place("red_cube", above("blue_cube"))
```

指示: すべての立方体を y=0 の線上に x 方向に 8cm 間隔で並べて。
```python
names = get_obj_names()
for i, n in enumerate(names):
    pick_place(n, np.array([TABLE["x_min"] + 0.05 + 0.08 * i, 0.0, TABLE["z"]]))
```

指示: 一番右（y が最小）にある立方体を中央に移動して。
```python
names = get_obj_names()
right = min(names, key=lambda n: get_obj_pos(n)[1])
pick_place(right, np.array([(TABLE["x_min"] + TABLE["x_max"]) / 2, 0.0, TABLE["z"]]))
```
'''

def extract_code(text):
    m = re.search(r"```python\n(.*?)```", text, re.S)
    return m.group(1) if m else text

def run_instruction(instruction, max_retry=2):
    messages = [{"role": "user", "content": PROMPT_HEADER + f"\n# 指示: {instruction}\n"}]
    for attempt in range(max_retry + 1):
        reply = ollama.chat(model=MODEL, messages=messages, options={"temperature": 0})
        code = extract_code(reply["message"]["content"])
        print(f"--- 生成コード (試行 {attempt}) ---\n{code}")
        allowed = {"np": np, "get_obj_names": get_obj_names, "get_obj_pos": get_obj_pos,
                   "pick_place": pick_place, "above": above, "TABLE": TABLE}
        try:
            exec(code, {"__builtins__": {"range": range, "len": len, "enumerate": enumerate,
                                         "min": min, "max": max, "sorted": sorted, "print": print,
```

```
                                        "abs": abs, "list": list, "zip": zip}}, allowed)
            return True
        except Exception:
            tb = traceback.format_exc(limit=2)
            print(tb)
            messages += [{"role": "assistant", "content": f"```python\n{code}```"},
                         {"role": "user", "content": f"実行時にエラーになった。修正したコードだけを出力して。\n{tb}"}]
    return False


if __name__ == "__main__":
    cam.start_recording()
    api.home()
    for inst in ["緑の立方体の上に赤を、その上に青を積んで、塔を作って。",
                 "塔を崩して、すべての立方体を y=0.2 の線上に並べ直して。"]:
        print(f"\n===== {inst} =====")
        ok = run_instruction(inst)
        print("成功" if ok else "失敗")
    api.home()
    cam.stop_recording(save_to_filename="cap_tower.mp4", fps=60)
```

プロンプトの中にコードブロック記号（バッククォート3つ）が含まれるため、このコードだけ `~~~` で囲んでいます。コピーするときは `~~~python` と `~~~` の行を含めないでください。

ここから細かく解説します。

- **API の設計**
- CaP の性能は、**どんな API を見せるか**でほぼ決まります。
- `above()` のような「よく使う空間計算」を用意しておくと、LLM が座標計算を間違えにくくなります。
- 一方で、API を増やしすぎるとプロンプトが長くなり、小さいモデルは混乱します。
- **Few-shot の例**
- 論文と同じく、「指示 → コード」の例をコメント形式で並べています。
- 例の中に**ループ・min/max・算術**を入れておくと、それを真似して複雑な指示にも対応します。
- **制限付きの `exec`**
- `__builtins__` を絞り、公開する名前を `allowed` だけにしています。`import os` などの危険なコードを防ぐためです。
- 完全な安全策ではないので、レポートの「実用性」で限界も書いておくとよいでしょう。
- **自己修正ループ**
- 論文の発展として、エラーのトレースバックを LLM に返して修正させます。小さいローカルLLM でもこれで成功率が上がります。

実行

```
python cap_genesis.py
```

実行結果の見方

- コンソールに、LLM が書いたコードが表示されます。動画に字幕として入れると、「AIが考えたプログラムでロボットが動く」ことが伝わります。

- `cap_tower.mp4` で、塔を作ってから一列に並べ直す様子が見られます。

比較実験（最終課題の本体）

1. **Few-shot の有無** - 例 0 個 / 1 個 / 3 個で、10個の指示に対する成功率を比べます。
2. **自己修正ループの有無** - エラーの再試行をすると、成功率がどれだけ上がるかを調べます。
3. **指示の難易度** - 次のように段階を分けて評価します。
    - 単純: 「AをBの上に」
    - 空間: 「三角形に並べて」
    - 論理: 「一番遠いものから順に積んで」
4. **安全性チェック** - `pick_place` の中で `TABLE` の範囲外の座標を拒否し、LLM に「範囲外です」と返します。

参考

- 論文: https://arxiv.org/abs/2209.07753
- コード: https://github.com/google-research/google-research/tree/master/code_as_policies
- プロジェクトページ: https://code-as-policies.github.io/
- 共通部品 franka_api.py: A07

目次へ

B06 SayCan: LLMのスコアと実行可能性を掛け合わせて行動を選ぶ

環境

- OS: Ubuntu 22.04（Colab 可）
- Python: 3.10
- シミュレータ: Genesis + Franka（A07 の `franka_api.py` を使います）
- LLM: Hugging Face Transformers のローカルモデル（例: `Qwen/Qwen2.5-1.5B-Instruct`）
- SayCan では、**各候補の文の対数尤度**が必要です。そのため、尤度を直接計算できる transformers を使います。
- 論文: Michael Ahn et al., "Do As I Can, Not As I Say: Grounding Language in Robotic Affordances", CoRL 2022
- コード（公式 Colab）: https://github.com/google-research/google-research/tree/master/saycan
- プロジェクトページ: https://say-can.github.io/

概要

論文のポイント

LLM は「何をすべきか」は知っていても、「**今このロボットに何ができるか**」は知りません。SayCan は、行動の候補（スキル）ごとに2つのスコアを掛け合わせます。

- **Say（LLMのスコア）**: 指示に対して、そのスキルが次の一手として**言語的にもっともらしいか**。候補の文の対数尤度で測ります。
- **Can（実行可能性 / アフォーダンス）**: 今の状態で、そのスキルが**成功しそうか**。論文では強化学習の価値関数で測っています。

スコアは `score = p_LLM(スキル | 指示, これまでの行動) × p_affordance(スキル | 状態)` で、最大のスキルを実行します。これを「done」が選ばれるまで繰り返します。

この記事でやること

- スキルの候補: 「X を掴んで Y の上に置く」（全組み合わせ）と「done」
- Say: Qwen2.5（1.5B）で各候補の文の対数尤度を計算します。
- Can: 論文の価値関数の代わりに、**シミュレータの状態から計算するルールベースのアフォーダンス**を使います。
- 掴む物体の上に何か載っていたら掴めない（0）。
- 置き先の上が空いていなければ置けない（0）。
- 腕が届かない距離なら低くする。
- 比較: **Say だけ**（LLM の言う通り）と **Say × Can**（SayCan）で、成功率がどう変わるかを見ます。

処理の流れは次のとおりです。

1. ユーザーが指示する。
2. スキルの候補（pick X, place on Y / done）を全列挙する。
3. 候補ごとに2つのスコアを出す。 - Say: LLM による対数尤度 - Can: 状態から計算するアフォーダンス
4. 2つの積が最大のスキルを実行する（FrankaAPI 経由）。
5. 「これまでの行動」に追記して、2に戻る。done が選ばれたら終了する。

準備

```
conda activate genesis
pip install transformers accelerate
```

ソースコード

saycan_genesis.py

```
import itertools
import numpy as np
import torch
from transformers import AutoModelForCausalLM, AutoTokenizer
from franka_api import FrankaAPI, make_scene, to_np

LLM_NAME = "Qwen/Qwen2.5-1.5B-Instruct"
tok = AutoTokenizer.from_pretrained(LLM_NAME)
llm = AutoModelForCausalLM.from_pretrained(LLM_NAME, torch_dtype=torch.float16, device_map="auto")
llm.eval()

CUBE = 0.04
OBJECTS = {
    "red block":    ((0.55,  0.15, 0.02), (1.0, 0.1, 0.1)),
    "blue block":   ((0.60, -0.10, 0.02), (0.1, 0.2, 1.0)),
    "yellow block": ((0.45,  0.00, 0.02), (1.0, 0.9, 0.1)),
}
PLACES = {"green bowl": (0.40, 0.30, 0.02), "gray bowl": (0.40, -0.30, 0.02)}  # 置き場（目印なし）

scene, franka, ents, cam = make_scene(OBJECTS)
api = FrankaAPI(scene, franka, cam)


def obj_pos(n):
    return to_np(ents[n].get_pos())


# ---- Say: LLM による候補の対数尤度 ----
FEWSHOT = """Robot: I can pick up a block and place it on another block or in a bowl.
Human: put the red block in the green bowl.
Robot: 1. pick up the red block and place it in the green bowl.
Robot: 2. done.
Human: stack the blue block on the yellow block, then put the red block on top.
Robot: 1. pick up the blue block and place it on the yellow block.
Robot: 2. pick up the red block and place it on the blue block.
Robot: 3. done.
"""
```

```
@torch.no_grad()
def llm_logprob(prompt, option):
    p_ids = tok(prompt, return_tensors="pt").input_ids
    full = tok(prompt + option, return_tensors="pt").input_ids.to(llm.device)
    logits = llm(full).logits[0, :-1].float()
    logp = torch.log_softmax(logits, dim=-1)
    target = full[0, 1:]
    start = p_ids.shape[1] - 1
    return logp[start:].gather(1, target[start:, None]).sum().item()


# ---- Can: 状態に基づくアフォーダンス ----
def something_on_top(name):
    p = obj_pos(name)
    for other in ents:
        if other == name:
            continue
        q = obj_pos(other)
        if np.linalg.norm(q[:2] - p[:2]) < CUBE * 0.7 and q[2] > p[2] + CUBE * 0.5:
            return True
    return False


def affordance(skill):
    if skill == "done.":
        return 1.0
    obj, dest = skill_args[skill]
    if something_on_top(obj):
        return 0.0                                  # 上に何か載っていて掴めない
    if dest in ents:
        if dest == obj or something_on_top(dest):
            return 0.0                              # 置き先がふさがっている
    reach = np.linalg.norm(obj_pos(obj)[:2])
    return 1.0 if reach < 0.75 else 0.2             # 遠いと成功率が下がる


# ---- スキル候補の列挙と実行 ----
skill_args = {}
for o, d in itertools.permutations(list(ents) + list(PLACES), 2):
    if o in ents:
        prep = "in the" if d in PLACES else "on the"
        skill_args[f"pick up the {o} and place it {prep} {d}."] = (o, d)
SKILLS = list(skill_args) + ["done."]


def execute(skill):
    o, d = skill_args[skill]
    target = np.array(PLACES[d]) if d in PLACES else obj_pos(d) + np.array([0, 0, CUBE])
    api.pick(obj_pos(o))
```

```
    api.place(target)


def saycan(instruction, use_affordance=True, max_steps=6):
    prompt = FEWSHOT + f"Human: {instruction}\n"
    for step in range(1, max_steps + 1):
        prefix = prompt + f"Robot: {step}. "
        say = np.array([llm_logprob(prefix, s) for s in SKILLS])
        say = np.exp(say - say.max()); say /= say.sum()   # 候補の中で正規化
        can = np.array([affordance(s) for s in SKILLS]) if use_affordance else np.ones(len(SKILLS))
        score = say * can
        best = SKILLS[int(score.argmax())]
        top3 = np.argsort(-score)[:3]
        print(f"[step {step}] " + " | ".join(f"{SKILLS[i]} say={say[i]:.2f} can={can[i]:.1f}" for i in top3))
        if best == "done.":
            break
        execute(best)
        prompt = prefix + best + "\n"
    return prompt


if __name__ == "__main__":
    cam.start_recording()
    api.home()
    # 事前に「黄色の上に赤」を積んでおき、アフォーダンスが効く状況を作る
    execute("pick up the red block and place it on the yellow block.")
    print(saycan("put the yellow block in the gray bowl.", use_affordance=True))
    api.home()
    cam.stop_recording(save_to_filename="saycan.mp4", fps=60)
```

ここから細かく解説します。

- `llm_logprob()`
- プロンプトの後ろに候補の文を付けた全体を LLM に通し、**候補部分のトークンの対数確率の和**を計算します。
- 論文の「LLM スコア」の再現です。トークンの区切りがプロンプトと候補の境目でずれることがありますが、比較には大きな影響はありません。
- **正規化**
- 候補の文の長さによって対数尤度の大きさが変わるので、候補の中で softmax を取っています。
- 長さの影響を減らしたい場合は、トークン数で割った平均の対数尤度も試してみるとよいでしょう（比較実験になります）。
- **アフォーダンスが効く例**
- 「黄色を灰色のボウルへ」という指示に対して、黄色の上には赤が載っています。
- Say だけだと、いきなり「黄色を掴む」を選んで失敗します。
- SayCan では、黄色の Can が 0 になるので、**まず赤をどかす**行動が選ばれます。これがこのデモの見どころです。
- ただし、赤をどかす行動が Say のスコアで上位に来るかはモデル次第です。Few-shot に「上に載っているものをどかす」例を足すと安定します。
- `PLACES`
- ボウルは目印として置いていないので、Genesis に円柱などを追加して見た目を付けると、動画がわかりやすくなります。

実行

```
python saycan_genesis.py
```

- `use_affordance=False` に変えて再実行すると、**Say だけ**の場合と比べられます。
- 各ステップの上位3候補（say / can の値）がログに出ます。そのままレポートに表として載せられます。

比較実験（最終課題の本体）

1. **Say だけ / SayCan の成功率** - 物体の積み方の初期状態を5パターン用意して比べます。
2. **LLM の大きさ** - 0.5B / 1.5B / 7B で比べます。大きいほど Say だけでも良くなるのか、Can の重要性は変わるのか、を考察します。
3. **学習したアフォーダンス** - ルールの代わりに「掴みが成功したか」をシミュレーションで100回試し、成功率を Can にします（データ駆動版）。

参考

- 論文: https://arxiv.org/abs/2204.01691
- 公式 Colab: https://github.com/google-research/google-research/tree/master/saycan
- プロジェクトページ: https://say-can.github.io/
- 共通部品 franka_api.py: A07

目次へ

B07 VoxPoser: LLMが作る3D価値マップで障害物を避けて動く

環境

- OS: Ubuntu 22.04
- Python: 3.10
- シミュレータ: Genesis + Franka（A07 の `franka_api.py` を使います）
- LLM: Ollama（`qwen2.5-coder:7b` など）
- 論文: Wenlong Huang et al., "VoxPoser: Composable 3D Value Maps for Robotic Manipulation with Language Models", CoRL 2023
- コード: https://github.com/huangwl18/VoxPoser
- 公式は RLBench + CoppeliaSim + OpenAI API が前提です。
- プロジェクトページ: https://voxposer.github.io/

概要

論文のポイント

VoxPoser は、LLM に**直接軌道を出させるのではなく**、作業空間を3Dのボクセル（小さな立方体の格子）に分け、その上の「**価値マップ**」**を作るコード**を書かせます。

- **アフォーダンスマップ**: 行きたい場所の価値を高くします（例: 「引き出しの取っ手」）。
- **回避マップ**: 避けたい場所のコストを高くします（例: 「花瓶の周り 10cm」）。
- その他にも、回転マップ、速度マップ、グリッパの開閉マップがあります。

LLM は「これらのマップをどう組み合わせるか」を書くだけで、**軌道はモーションプランナーがマップの上で探索**します。そのため、追加の学習なし（ゼロショット）で、「花瓶を避けて」「ゆっくり」といった制約付きの指示に対応できます。

この記事でやること

VoxPoser の考え方を、**Genesis で小さく再実装**します。

- 作業空間: x ∈ [0.25, 0.75]、y ∈ [-0.35, 0.35]、z ∈ [0, 0.4] を 2cm のボクセルに分けます（25 × 35 × 20）。
- LLM に書かせるのは、`affordance` と `avoidance` を組み立てるコードです。
- プランナー: コストマップ上の **A*** で経路を探し、間引いた経由点を `FrankaAPI.move_to` でたどります。
- 指示の例: 「赤い立方体を掴んで、背の高い花瓶にぶつからないように右側の置き場へ運んで」

処理の流れは次のとおりです。

1. ユーザーが自然言語で指示する（例: 「花瓶を避けて赤を右へ」）。
2. LLM がマップを作るコードを生成する。 - 例: `affordance = target_map('right_zone')`、`avoidance = avoid_map('vase', 0.10)`
3. コストマップを作る: `cost = 1 - 正規化した affordance + 10 × avoidance`
4. A* で現在の手先位置から、affordance が最大の点まで経路を探す。
5. 経由点を間引いて、FrankaAPI.move_to で順番にたどる。

準備

```
conda activate genesis
pip install ollama scipy
ollama pull qwen2.5-coder:7b
```

（任意）公式実装

```
git clone https://github.com/huangwl18/VoxPoser.git
```

- RLBench のインストール手順に従い、CoppeliaSim と PyRep を入れます。
- `src/playground.ipynb` を開き、OpenAI のキーを設定します。
- 公式のプロンプト（`src/prompts/rlbench/*.txt`）は、LLM に書かせるコードの例として**読むだけでも非常に参考になります**。

ソースコード

voxposer_mini.py

```
import heapq
import re
import numpy as np
import ollama
import genesis as gs
from scipy.ndimage import distance_transform_edt
from franka_api import FrankaAPI, to_np

MODEL = "qwen2.5-coder:7b"
RES = 0.02
LO = np.array([0.25, -0.35, 0.0])
HI = np.array([0.75, 0.35, 0.40])
SHAPE = tuple(((HI - LO) / RES).astype(int))   # (25, 35, 20)

# ---- シーン（花瓶 = 背の高い円柱を障害物として置く） ----
gs.init(backend=gs.gpu)
scene = gs.Scene(sim_options=gs.options.SimOptions(dt=0.01), show_viewer=False)
scene.add_entity(gs.morphs.Plane())
red = scene.add_entity(gs.morphs.Box(size=(0.04, 0.04, 0.04), pos=(0.55, 0.20, 0.02)),
                       surface=gs.surfaces.Default(color=(1, 0.1, 0.1)))
vase = scene.add_entity(gs.morphs.Cylinder(radius=0.04, height=0.30, pos=(0.55, 0.0, 0.15), fixed=True),
                        surface=gs.surfaces.Default(color=(0.6, 0.3, 0.8)))
franka = scene.add_entity(gs.morphs.MJCF(file="xml/franka_emika_panda/panda.xml"))
cam = scene.add_camera(res=(960, 540), pos=(1.4, 0.9, 0.9), lookat=(0.5, 0, 0.1), fov=45, GUI=False)
scene.build()
api = FrankaAPI(scene, franka, cam)

NAMED = {"red_cube": red, "vase": vase}
ZONES = {"right_zone": np.array([0.55, -0.25, 0.02]), "left_zone": np.array([0.55, 0.25, 0.02])}
```

```
# ---- ボクセル変換と、LLM に公開するマップ関数 ----
def to_idx(p):
    return tuple(np.clip(((np.asarray(p) - LO) / RES).astype(int), 0, np.array(SHAPE) - 1))


def to_pos(idx):
    return LO + (np.array(idx) + 0.5) * RES


def get_pos(name):
    """物体名またはゾーン名の中心座標を返す"""
    return ZONES[name].copy() if name in ZONES else to_np(NAMED[name].get_pos())


def target_map(name, z_offset=0.0):
    """name の位置（+ z_offset）を最大値1とするアフォーダンスマップ（ガウス分布）"""
    m = np.zeros(SHAPE)
    m[to_idx(get_pos(name) + np.array([0, 0, z_offset]))] = 1
    d = distance_transform_edt(1 - m) * RES
    return np.exp(-(d / 0.04) ** 2)


def avoid_map(name, margin):
    """name の周り margin[m] 以内を1にする回避マップ（花瓶は z 方向にも高さ分の広がりを持つ）"""
    m = np.zeros(SHAPE)
    p = get_pos(name)
    xs, ys, zs = np.meshgrid(*[np.arange(s) for s in SHAPE], indexing="ij")
    pts = LO + (np.stack([xs, ys, zs], -1) + 0.5) * RES
    horiz = np.linalg.norm(pts[..., :2] - p[:2], axis=-1)
    m[(horiz < margin + 0.04) & (pts[..., 2] < p[2] * 2 + margin)] = 1
    return m


# ---- A* でコストマップ上の経路を探す ----
def astar(cost, start, goal):
    moves = [(dx, dy, dz) for dx in (-1, 0, 1) for dy in (-1, 0, 1) for dz in (-1, 0, 1) if (dx, dy, dz) != (0, 0, 0)]
    h = lambda a: np.linalg.norm(np.subtract(a, goal))
    openq, came, g = [(h(start), start)], {}, {start: 0.0}
    while openq:
        _, cur = heapq.heappop(openq)
        if cur == goal:
            path = [cur]
            while cur in came:
                cur = came[cur]
                path.append(cur)
            return path[::-1]
        for mv in moves:
            nb = tuple(np.add(cur, mv))
            if any(c < 0 or c >= s for c, s in zip(nb, SHAPE)):
```

```
                continue
            ng = g[cur] + np.linalg.norm(mv) * (1 + cost[nb])
            if ng < g.get(nb, np.inf):
                g[nb], came[nb] = ng, cur
                heapq.heappush(openq, (ng + h(nb), nb))
    return None


def plan_and_move(affordance, avoidance, z_lift=0.12):
    cost = 10.0 * avoidance
    goal_pos = to_pos(np.unravel_index(np.argmax(affordance), SHAPE)) + np.array([0, 0, z_lift])
    start_pos = to_np(api.ee.get_pos())
    path = astar(cost, to_idx(start_pos), to_idx(goal_pos))
    if path is None:
        raise RuntimeError("経路が見つかりません")
    waypoints = [to_pos(p) for p in path[::4]] + [goal_pos]
    for w in waypoints:
        api.move_to(w, steps=25)
    return goal_pos


# ---- LLM にマップを組み立てるコードを書かせる ----
PROMPT = '''
# 3D価値マップでロボットの動きを決める。次の関数が使える。
# target_map(name, z_offset=0.0) -> 行きたい場所のマップ
# avoid_map(name, margin) -> 避けたい場所のマップ（margin は m）
# 物体名: "red_cube", "vase"  ゾーン名: "right_zone", "left_zone"
# 2段階のサブタスクに分け、各サブタスクで affordance と avoidance を代入するコードを書く。
# 例
# 指示: 赤い立方体を左ゾーンへ運んで。
```python
subtasks = []
# 1. 赤い立方体の真上へ
affordance = target_map("red_cube"); avoidance = 0 * target_map("red_cube")
subtasks.append(("grasp", affordance, avoidance))
# 2. 左ゾーンへ
affordance = target_map("left_zone"); avoidance = 0 * target_map("left_zone")
subtasks.append(("release", affordance, avoidance))
```
'''

def compose(instruction):
    r = ollama.chat(model=MODEL, messages=[{"role": "user", "content": PROMPT + f"# 指示: {instruction}\n"}],
                    options={"temperature": 0})
    code = re.search(r"```python\n(.*?)```", r["message"]["content"], re.S).group(1)
    print(code)
    ns = {"target_map": target_map, "avoid_map": avoid_map, "np": np}
    exec(code, ns)
```

```
    return ns["subtasks"]


if __name__ == "__main__":
    cam.start_recording()
    api.home()
    subtasks = compose("赤い立方体を掴んで、花瓶から10cm以上離れて右ゾーンに運んで。")
    for kind, aff, avd in subtasks:
        goal = plan_and_move(aff, avd)
        if kind == "grasp":
            api.open_gripper()
            grasp = get_pos("red_cube") + np.array([0, 0, FrankaAPI.GRASP_OFFSET])
            api.move_to(grasp, steps=60)          # 掴む高さへ下降
            api.close_gripper()
            api.move_to(goal, steps=60)
        else:
            api.move_to(goal - np.array([0, 0, 0.08]), steps=60)   # 少し下げてから離す
            api.open_gripper()
            api.move_to(goal, steps=40)
    api.home()
    cam.stop_recording(save_to_filename="voxposer_mini.mp4", fps=60)
```

プロンプトの中にコードブロック記号（バッククォート3つ）が含まれるため、このコードだけ `~~~` で囲んでいます。コピーするときは `~~~python` と `~~~` の行を含めないでください。

ここから細かく解説します。

- **ボクセルの価値マップ**
- `target_map` は、目標の1点から距離に応じて減衰するガウス分布です。
- `avoid_map` は、障害物の周りを1にした二値のマップです。
- LLM はこれを**組み合わせる**だけなので、座標計算を自分でする必要がありません。これが VoxPoser の「composable（組み合わせ可能）」の意味です。
- **コストの設計**
- `cost = 10 × avoidance` を移動コストに掛けます。そのため、A* は花瓶の周りを大きく迂回します。
- 回避の重み 10 を変えると、「どれくらい避けるか」が変わります（比較実験になります）。
- **A* の経路の間引き**
- 2cm 刻みの経路をそのまま IK でたどると遅いので、4点ごとに間引いています。
- **論文との違い**
- 論文には、VLM（OWL-ViT + SAM）で物体の位置を画像から求める部分や、MPC による閉ループ制御もあります。今回は位置をシミュレータから直接取っています。レポートにはこの簡略化を必ず書いてください。

実行

```
python voxposer_mini.py
```

実行結果の見方

- 手先が花瓶をぐるっと回り込んで右ゾーンへ向かえば成功です。
- 価値マップを matplotlib で可視化すると、レポートの図として非常にわかりやすくなります。

```
import matplotlib.pyplot as plt
plt.imshow(avoid_map("vase", 0.10)[:, :, 5].T, origin="lower"); plt.title("avoidance (z=0.1m)")
plt.savefig("avoid_map.png")
```

比較実験（最終課題の本体）

1. **回避マップの有無** - なしの場合は直線的に進むので花瓶に当たる、という違いを動画で並べます。
2. **margin を言葉で変える** - 「ギリギリを通って」「大きく避けて」という言い方の違いで、LLM が書く `margin` がどう変わり、経路がどう変わるかを見ます。
3. **論文の他のマップ** - 速度マップ（「花瓶の近くではゆっくり」）を追加し、`move_to` の `steps` を変えます。

参考

- 論文: https://arxiv.org/abs/2307.05973
- コード: https://github.com/huangwl18/VoxPoser
- プロジェクトページ: https://voxposer.github.io/
- 共通部品 franka_api.py: A07

目次へ

B08 CLIPortで言語指示による積み替えと未知色への汎化を評価する

環境

- OS: Ubuntu 20.04 / 22.04
- Python: 3.8（公式 README の指定）
- シミュレータ: PyBullet（Ravens ベンチマーク）
- GPU: VRAM 8GB以上（評価のみ）、学習は 12GB以上を推奨
- 論文: Mohit Shridhar, Lucas Manuelli, Dieter Fox, "CLIPort: What and Where Pathways for Robotic Manipulation", CoRL 2021
- コード: https://github.com/cliport/cliport
- プロジェクトページ: https://cliport.github.io/

概要

論文のポイント

CLIPort は、言語で指示された**ピック＆プレース**を、真上から見た画像の上で解きます。

- **2つの経路（Two-Stream）**
- **What 経路（意味）**: CLIP の視覚・言語特徴で「何を」を理解します（例: 「青いブロック」）。
- **Where 経路（空間）**: Transporter Networks で「どこに、どの向きで」を正確に決めます。
- 出力は「掴む画素」と「置く画素＋回転角」です。
- CLIP の事前知識のおかげで、**学習時に見ていない色や物体（unseen）にもある程度汎化します。**

この記事でやること

1. 公式の学習済みモデルをダウンロードして、すぐに評価を動かす。
2. 「seen（学習時に見た色）」と「unseen（見ていない色）」のタスクで成功率を比べる。
3. 自分で少ないデモで学習し、デモ数と成功率の関係を調べる。

準備

```
git clone https://github.com/cliport/cliport.git
cd cliport
python3.8 -m venv cliport_env      # README では virtualenv を使用
source cliport_env/bin/activate
pip install --upgrade pip
pip install -r requirements.txt
export CLIPORT_ROOT=$(pwd)
python setup.py develop
```

- `export CLIPORT_ROOT=...` は、新しい端末を開くたびに必要です。`~/.bashrc` に書いておくとよいでしょう。
- requirements の torch の版が古いので、手元の CUDA と合わない場合は、torch だけ CUDA に合う版を入れ直します。

学習済みモデルのダウンロード（Quickstart）

```
sh scripts/quickstart_download.sh
```

実行1: 学習済みモデルの評価

評価用のデモ（初期配置）を生成

```
python cliport/demos.py n=10 task=packing-boxes-pairs-seen-colors mode=test
```

評価（GUI 付き）

```
python cliport/eval.py model_task=multi-language-conditioned \
  eval_task=packing-boxes-pairs-seen-colors \
  agent=cliport mode=test n_demos=10 train_demos=1000 \
  exp_folder=cliport_quickstart checkpoint_type=test_best \
  update_results=True disp=True
```

ここから細かく解説します。

- `model_task=multi-language-conditioned`
- 複数のタスクをまとめて学習したモデルです。
- `eval_task`
- 評価するタスクです。`-seen-colors` / `-unseen-colors` の接尾辞で、色の汎化を評価できます。
- `disp=True`
- PyBullet の GUI を表示します。画面録画の素材になります。
- 動画ファイルとして保存したい場合は `record.save_video=True` を付けます（`cliport/cfg/eval.yaml` の `record` 項目を参照）。
- **結果**
- `exp_folder` の下に成功率を記録した JSON が保存されます。

実行2: seen / unseen の比較（最終課題の本体）

```
for t in stack-block-pyramid-seq-seen-colors stack-block-pyramid-seq-unseen-colors \
         packing-boxes-pairs-seen-colors packing-boxes-pairs-unseen-colors \
         put-block-in-bowl-seen-colors put-block-in-bowl-unseen-colors; do
  python cliport/demos.py n=20 task=$t mode=test
  python cliport/eval.py model_task=multi-language-conditioned eval_task=$t \
    agent=cliport mode=test n_demos=20 train_demos=1000 \
    exp_folder=cliport_quickstart checkpoint_type=test_best update_results=True disp=False
done
```

- タスクごとに seen と unseen の成功率を並べたグラフを作ります。
- 「unseen で下がるが0にはならない」という結果が出れば、CLIP の汎化の効果として考察できます。

実行3: 少ないデモで自分で学習する

学習用デモの生成

```
python cliport/demos.py n=100 task=put-block-in-bowl-seen-colors mode=train
python cliport/demos.py n=20  task=put-block-in-bowl-seen-colors mode=val
python cliport/demos.py n=20  task=put-block-in-bowl-seen-colors mode=test
```

学習

```
python cliport/train.py train.task=put-block-in-bowl-seen-colors \
  train.agent=cliport train.attn_stream_fusion_type=add train.trans_stream_fusion_type=conv \
  train.lang_fusion_type=mult train.n_demos=100 train.n_steps=20000 \
  train.exp_folder=exps_mine dataset.cache=False
```

- `train.n_demos` を 10 / 50 / 100 と変えて学習し、デモ数と成功率の関係を調べます。
- 引数の名前は `cliport/cfg/train.yaml` で確認できます。

自分で学習したモデルの評価

```
python cliport/eval.py model_task=put-block-in-bowl-seen-colors \
  eval_task=put-block-in-bowl-unseen-colors agent=cliport mode=test n_demos=20 \
  train_demos=100 exp_folder=exps_mine checkpoint_type=val_missing update_results=True disp=True
```

発展: 自分の指示文を試す

`cliport/tasks/` の中のタスクファイルでは、言語テンプレート（例: `"put the {pick} block in the {place} bowl"`）が定義されています。これを書き換えて、次のような言い換えを試せます。

- 同じ意味の言い換え（"move the red cube into the green container"）
- 日本語

CLIP の言語エンコーダは英語中心なので、日本語では性能が大きく落ちるはずです。それを確認すること自体が考察になります。

工夫ポイント（最終課題として発展させる案）

1. **seen と unseen の差の定量化**（上記の実行2）
2. **アブレーション** - `agent=transporter`（言語なし）や `agent=two_stream_clip_lingunet_lat_transporter` など、論文の比較手法を評価します。
3. **言い換えへの頑健性** - テンプレートを変えた指示での成功率を測ります。
4. **注意マップの可視化** - 掴む位置と置く位置の affordance ヒートマップを画像として保存すると、「AIがどこを見ているか」を図で示せます。

参考

- 論文: https://arxiv.org/abs/2109.12098
- コード: https://github.com/cliport/cliport
- プロジェクトページ: https://cliport.github.io/
- Transporter Networks: Zeng et al., CoRL 2020

- CLIP: Radford et al., ICML 2021

目次へ

B09 OpenVLAをLIBEROで評価する

環境

- OS: Ubuntu 22.04
- Python: 3.10
- シミュレータ: LIBERO（robosuite / MuJoCo ベース）
- モデル: OpenVLA-7B（LIBERO で微調整済みのチェックポイント）
- GPU:
- bf16 で推論する場合: VRAM 16GB以上
- 4bit 量子化する場合: 約8GB
- Colab の A100 / L4 も使えます。
- 論文: Moo Jin Kim et al., "OpenVLA: An Open-Source Vision-Language-Action Model", CoRL 2024
- コード:
- https://github.com/openvla/openvla
- https://github.com/Lifelong-Robot-Learning/LIBERO
- プロジェクトページ: https://openvla.github.io/

概要

論文のポイント

OpenVLA は、Llama 2（7B）をベースにした視覚言語モデル（Prismatic VLM。視覚には SigLIP と DINOv2 を併用）を、Open X-Embodiment の約97万エピソードのロボットデータで学習した **VLA（Vision-Language-Action）モデル**です。

- 入力は「カメラ画像 + 言語指示」です。
- 出力は「7次元の行動」（手先の xyz・回転の変化量とグリッパ）です。
- **行動を256段階に離散化し、LLM のトークンとして出力する**のが特徴です（RT-2 と同じ考え方）。
- 公開されている VLA の中で、当時の RT-2-X（55B）を上回る性能を、7B のモデルで達成しました。

この記事でやること

1. 公式の評価スクリプトで、LIBERO の4つのタスク群（suite）を評価する。
2. **言語指示を書き換えて**、汎化の限界を調べる（最終課題の本体）。
3. 失敗動画を分類して考察する。

LIBERO のタスク群は次の4つです。

- **libero_spatial**: 同じ物体で、配置だけが違う。
- **libero_object**: 違う物体を掴む。
- **libero_goal**: 同じ配置で、目標が違う。
- **libero_10（long）**: 長い手順のタスク。

準備

```
conda create -n openvla python=3.10 -y
conda activate openvla
```

```
conda install pytorch torchvision torchaudio pytorch-cuda=12.4 -c pytorch -c nvidia -y

git clone https://github.com/openvla/openvla.git
cd openvla
pip install -e .
# 学習もする場合（推論だけなら任意）
pip install packaging ninja
pip install "flash-attn==2.5.5" --no-build-isolation

# LIBERO
git clone https://github.com/Lifelong-Robot-Learning/LIBERO.git
cd LIBERO && pip install -e . && cd ..
pip install -r experiments/robot/libero/libero_requirements.txt
```

- 初回に LIBERO を import すると、データの保存先を聞かれることがあります。Enter を押して既定値で構いません。
- 描画のエラーが出る場合は、`export MUJOCO_GL=egl` を設定します。

実行1: 公式の評価

```
python experiments/robot/libero/run_libero_eval.py \
  --model_family openvla \
  --pretrained_checkpoint openvla/openvla-7b-finetuned-libero-spatial \
  --task_suite_name libero_spatial \
  --center_crop True
```

ここから細かく解説します。

- `--pretrained_checkpoint`
- suite ごとに微調整済みのモデルがあります。
  - `openvla-7b-finetuned-libero-spatial`
  - `...-object`
  - `...-goal`
  - `...-10`
- `--center_crop True`
- 学習時にランダムクロップのデータ拡張をしているので、評価時は中央を切り出します。README で指定されている設定です。
- **VRAM が足りない場合**
- `--load_in_4bit True` を付けます。精度は少し落ちます。
- **出力**
- `./experiments/logs/` に成功率のログが出ます。
- `./rollouts/` にエピソードごとの mp4 が出ます。
- 1つの suite は10タスク×50試行です。時間がかかるので、課題では試行回数を減らします（`--num_trials_per_task 10`）。

実行2: 言語指示を書き換える（最終課題の本体）

`experiments/robot/libero/run_libero_eval.py` の中で、タスクの説明（`task_description`）を取得してモデルに渡しています。その直後に**書き換え処理**を入れます。

```
# run_libero_eval.py の、各タスクのループ内（task_description を取得した直後）に追加
PARAPHRASE = {
    # 元の指示 -> 言い換え（自分で作る）
    "pick up the black bowl between the plate and the ramekin and place it on the plate":
        "grab the dark bowl located between the dish and the small cup, then put it onto the dish",
}
MODE = os.environ.get("LANG_MODE", "original")   # original / paraphrase / wrong / empty
if MODE == "paraphrase":
    task_description = PARAPHRASE.get(task_description, task_description)
elif MODE == "wrong":
    task_description = "pick up the ketchup and place it in the basket"   # わざと別のタスク
elif MODE == "empty":
    task_description = ""
```

ファイルの先頭に `import os` が必要です。実行は次のとおりです。

```
for m in original paraphrase wrong empty; do
  LANG_MODE=$m python experiments/robot/libero/run_libero_eval.py \
    --model_family openvla --pretrained_checkpoint openvla/openvla-7b-finetuned-libero-spatial \
    --task_suite_name libero_spatial --center_crop True --num_trials_per_task 10 \
    --run_id_note lang_$m
done
```

考察のポイント

- **paraphrase で成功率が大きく下がる場合**: モデルは言語の意味ではなく、**決まった文字列に過剰適合している**可能性があります。
- **wrong や empty でも成功率が高い場合**: libero_spatial では、**画像だけで行動が決まってしまう**（言語を見ていない）ことを示します。
- この2つは、VLA 研究で実際に議論されている論点です。小さな実験でも、論文に基づいた考察になります。

実行3: 失敗の分類

`./rollouts/` の動画を見て、失敗を次のように分類し、割合を出します。

- 掴み損ね（グリッパの位置ずれ）
- 違う物体を掴んだ（言語の取り違え）
- 運搬中に落とした
- 時間切れ（迷って動かない）

工夫ポイント（最終課題として発展させる案）

1. **suite 間の比較**: spatial / object / goal / 10 で成功率を比べ、どの種類の汎化が難しいかを考察します。
2. **量子化の影響**: bf16 と 4bit で成功率と推論速度を比べます。
3. **OpenVLA-OFT との比較**: 改良版の OpenVLA-OFT（並列デコードと連続値の行動）が公開されています。速度と性能を比べます。
4. **π0 / SmolVLA との比較**: B10 の π0 や A06 の SmolVLA と、同じ suite で比べます。

参考

- 論文: https://arxiv.org/abs/2406.09246
- コード: https://github.com/openvla/openvla
- プロジェクトページ: https://openvla.github.io/
- LIBERO: https://github.com/Lifelong-Robot-Learning/LIBERO
- Open X-Embodiment: https://robotics-transformer-x.github.io/

目次へ

B10 π0（openpi）をLIBEROで動かしてOpenVLAと比較する

環境

- OS: Ubuntu 22.04（openpi は Ubuntu 22.04 でのみテストされています）
- パッケージ管理: uv（openpi の公式の手順）、または Docker
- シミュレータ: LIBERO
- モデル: π0 / π0-FAST（LIBERO で微調整済みのチェックポイント）
- GPU: 推論に VRAM 8GB以上（RTX 4090 クラスを推奨）。微調整には 24GB以上（LoRA の場合）が必要です。
- 論文:
- Kevin Black et al., "π0: A Vision-Language-Action Flow Model for General Robot Control", 2024
- Pertsch et al., "FAST: Efficient Action Tokenization for Vision-Language-Action Models", 2025
- コード: https://github.com/Physical-Intelligence/openpi

概要

論文のポイント

π0 は Physical Intelligence 社の VLA です。

- **視覚言語モデル（PaliGemma 3B）** に、**Flow Matching で連続値の行動を生成するアクションエキスパート**を組み合わせています。
- OpenVLA（B09）のように行動をトークンに離散化するのではなく、**連続値の行動列（action chunk）を 50Hz で滑らかに出力**します。洗濯物をたたむような器用な作業に強いのが特徴です。
- π0-FAST は、行動を周波数領域で圧縮してトークン化する「FAST」を使った自己回帰版です。

この記事でやること

openpi は「**ポリシーサーバ**」と「**ロボット（シミュレーション）クライアント**」を分ける構成です。

1. ターミナル1で、π0 の LIBERO 用チェックポイントを読み込んだポリシーサーバを起動する。
2. ターミナル2で、LIBERO のシミュレーションを動かすクライアントを起動する。
3. OpenVLA（B09）と同じ suite で成功率と推論速度を比べる（最終課題の本体）。

処理の流れは次のとおりです。

1. LIBERO クライアント（examples/libero/main.py）が、画像・状態・指示を WebSocket でポリシーサーバに送る。
2. ポリシーサーバ（scripts/serve_policy.py、π0 チェックポイント）が、行動列を返す。

このように分かれているので、**実機でもシミュレーションでも同じサーバが使えます。**この設計自体もレポートで説明すると良いポイントです。

準備

```
# uv のインストール（未導入の場合）
curl -LsSf https://astral.sh/uv/install.sh | sh

git clone --recurse-submodules https://github.com/Physical-Intelligence/openpi.git
cd openpi
```

```
GIT_LFS_SKIP_SMUDGE=1 uv sync
GIT_LFS_SKIP_SMUDGE=1 uv pip install -e .
```

- `--recurse-submodules` を忘れると、LIBERO などのサブモジュールが空になります。忘れた場合は `git submodule update --init --recursive` を実行します。

実行1: Docker で一発起動（おすすめ）

```
# NVIDIA Container Toolkit が必要
export SERVER_ARGS="--env LIBERO"
docker compose -f examples/libero/compose.yml up --build
```

サーバとクライアントが両方起動し、評価が始まります。

実行2: Docker を使わない場合

ターミナル1: LIBERO クライアント（Python 3.8 の別環境）

```
uv venv --python 3.8 examples/libero/.venv
source examples/libero/.venv/bin/activate
uv pip sync examples/libero/requirements.txt third_party/libero/requirements.txt \
  --extra-index-url https://download.pytorch.org/whl/cu113 --index-strategy=unsafe-best-match
uv pip install -e packages/openpi-client
uv pip install -e third_party/libero
export PYTHONPATH=$PYTHONPATH:$PWD/third_party/libero
python examples/libero/main.py
```

ターミナル2: ポリシーサーバ

```
uv run scripts/serve_policy.py --env LIBERO
```

ここから細かく解説します。

- `--env LIBERO`
- LIBERO 用の既定のチェックポイント（π0 系を LIBERO で微調整したもの）をクラウドから自動でダウンロードして起動します。
- 初回は数GBのダウンロードがあります。
- **使うチェックポイントの指定**
- 次のように明示できます。

```
uv run scripts/serve_policy.py policy:checkpoint \
  --policy.config=pi0_fast_libero \
  --policy.dir=gs://openpi-assets/checkpoints/pi0_fast_libero
```

- `pi0_libero`（Flow Matching 版）と `pi0_fast_libero`（FAST 版）を切り替えると、**同じモデル系統の2方式の比較**ができます。
- 設定名は版によって変わることがあるので、`src/openpi/training/config.py` で確認してください。
- **クライアントの設定**

- `examples/libero/main.py` の引数（`--args.task-suite-name libero_spatial` や `--args.num-trials-per-task 10`）で、suite と試行回数を変えられます。
- 動画は `data/libero/videos` に保存されます。

Python から直接推論する

```
from openpi.training import config as _config
from openpi.policies import policy_config
from openpi.shared import download

cfg = _config.get_config("pi0_fast_libero")
ckpt = download.maybe_download("gs://openpi-assets/checkpoints/pi0_fast_libero")
policy = policy_config.create_trained_policy(cfg, ckpt)

example = {
    "observation/image": agent_view_img,        # (224, 224, 3) uint8
    "observation/wrist_image": wrist_img,
    "observation/state": state,                 # (8,) 手先姿勢 + グリッパ
    "prompt": "put the bowl on the plate",
}
actions = policy.infer(example)["actions"]      # (action_horizon, 7)
```

入力キーの名前は `src/openpi/policies/libero_policy.py` に定義されています。

比較実験（最終課題の本体）

1. **π0 / π0-FAST / OpenVLA（B09）の比較** - 同じ suite（例: libero_spatial、各タスク10試行）で比べます。 - 比べる項目は、成功率と、1回の推論時間（ms）です。サーバのログに出るか、自分で `time.perf_counter()` で計測します。
2. **言語の書き換え** - B09 と同じ書き換え（paraphrase / wrong / empty）を `main.py` の `"prompt": str(task_description)` の部分で行い、言語への依存度を比べます。
3. **アクションチャンクの効果** - `main.py` の `replan_steps`（チャンクのうち何ステップ実行してから再推論するか）を 1 / 5 / 10 に変えて、性能と速度のトレードオフを調べます。ACT（B01）の議論ともつながります。

工夫ポイント（レポートでの書き方）

- 「サーバとクライアントの分離」「Flow Matching と離散トークンの違い」の2つを図で説明します。最新の VLA の設計を理解していることを示せます。
- 計算資源の制約で試行回数を減らした場合は、その旨と、信頼区間（例: 成功率 ± 標準誤差）を書きます。

参考

- openpi: https://github.com/Physical-Intelligence/openpi
- π0 論文: https://arxiv.org/abs/2410.24164
- FAST 論文: https://arxiv.org/abs/2501.09747
- Physical Intelligence ブログ: https://www.physicalintelligence.company/blog/pi0
- LIBERO: https://github.com/Lifelong-Robot-Learning/LIBERO
- 比較対象: B09 OpenVLA

目次へ